Typing Rules Become Programs in Tree Calculus

Draft of November 5, 2026

Barry Jay
Barry.Jay8@gmail.com

Abstract

In tree calculus, program analyses can be internalised as programs in the calculus itself. Here, typing rules become programs. In this combinatory approach, the traditional function types, type variables and quantifiers are displaced by type application. The soundness and expressive power of the type system are determined by an ontology of abstract types. This can type general recursion, polymorphic lists, type-directed conditionals, length-indexed arrays, and queries and updates, both generic and specialised. Even the type inference program is typed: it infers its own type, which is itself. All theorems have been mechanised in Rocq.

1 Introduction

Traditionally, typing is expressed by judgements. For example, $\Gamma \vdash t : T$ asserts that the term t has type T in context Γ [4, 5, 9, 15, 36]. These judgements are derived by rules that live outside the term language, that must analyse the types of their premises, check their equality, and so on. Moreover, types are commonly stratified into a hierarchy of types, kinds or sorts [4, 14, 32]. This paper keeps typing within the one notion of computation of *tree calculus* [21] by defining a program infer so that the judgement $t : T$ becomes the evaluation

$$\mathsf{infer}\ @\ t \Longrightarrow \mathsf{Sm}\ T$$

where the stem Sm marks success. Since $\mathsf{Sm}\ T$ is the result of an evaluation, T is a value, a binary tree.

Since tree calculus is combinatory, typing is driven by type application, which computes the type of an application from the types of its parts and is written (#). Figure 1 shows six typed programs and how their types apply. General recursion on numbers can be built on nat_search which takes a predicate P and a starting point (say 0) and finds the first number that satisfies the predicate, if any. For example, nat_search @ is_odd $@_r$ 0 $\Longrightarrow$ 1 finds the first odd number. In a functional setting the search would take type (Nat → Bool) → (Nat → Nat) but here nat_search is *self-typed*, that is, it is its own type. The information is carried by the type applications nat_search # P # Nat $\Longrightarrow$ Sm Nat. Similar remarks apply to the append of lists. Their polymorphism is revealed by type application, where System F would quantify a type variable [15]. Further, type application can check that the inner dimensions of two matrices agree, to support a limited form of type dependency [40]. The fourth example typed_equal is of *query* type. It can decide equality of any pair of programs by inspecting both, whatever their

```
nat_search : nat_search
  nat_search # P # Nat ⟹ Sm Nat    (if P # Nat ⟹ Sm Bool)
  nat_search @ is_odd @_r 0 ⟹ 1

append : append
  append # (List[A] * List[A]) ⟹ Sm List[A]

matrix_mult : matrix_mult
  matrix_mult # (Matrix[m, n] * Matrix[n, p]) ⟹ Matrix[m, p]

typed_equal : Query[Query[Bool]]
  Query[Query[Bool]] # U # V ⟹ Sm Bool
  typed_equal @ typed_equal @_r typed_equal ⟹ tt

salary_update : Update
  Update # U ⟹ Sm U
  salary_update @ paid_over[57] ⟹ paid_over[58]
```

Figure 1. Six typed programs, with the type applications (#), typed applications (::) and evaluations that reveal their polymorphism

types U and V. In a traditional account, Query[Bool] could be represented by $\forall X.X \to \mathsf{Bool}$ but then type inference would need annotations, and parametricity would forbid inspecting the argument. The fifth example salary_update is an *update* that uses a type-directed conditional to increment the salaries it reaches, and leaves all other structures unchanged. It can be applied to salaries or departments full of employees or even programs that mention salaries, such as paid_over[57] : Query[List[Emp]]. So one update acts on both data and the queries that read it.

That type application displaces function types deserves some explanation. Consider the traditional rule for typing applications:

$$\frac{f : U \to V \qquad u : U}{f\ u : V}.$$

To apply it, one must recognise the type of f as a function type, recover its domain and codomain, then check that the domain equals the type of u. For example, the identity function in System F [15] has the quantified type $\forall X.\, X \to X$. One must instantiate this type variable before the rule can be applied. Similarly, booleans could be typed by $\forall X.(X * X) \to X$. Their application to some $u : A * B$ requires that A and B be equal, by comparing quantified types or by unification. This analysis of types is computation, yet in traditional type theory it is performed outside the term calculus. This creates a burden which we have learned to bear, but it also challenges the idea that there is only one notion of computation. An encoding of types as terms could be used to bridge the

gap but then the encoding would be one more computation to be realised. A satisfactory solution must make the types themselves be terms, so that typing analyses them as they stand. However, this cannot be done in, say, lambda calculus, since its abstractions can only be applied, not analysed: one cannot look under the lambda.

Combinatory logic may not seem to be any more promising. Indeed, combinators are usually typed by considering them as lambda terms. However, combinators support a different approach, namely *combinatory typing*, which is driven by

$$\frac{f : F \qquad u : U \qquad F \,\#\, U \Longrightarrow \mathsf{Sm}\, V}{f \mathbin{@} u :: V}$$

where the *type application* $F \,\#\, U$ computes the result type, if any, of applying a program of type F to a value of type U. The application $f \mathbin{@} u$ is typed as an expression, not as a program, so the double colon :: records its type. For example, the traditional application using function types is obtained by ensuring that a function type applied to its domain satisfies

$$(U \to V) \,\#\, U \Longrightarrow \mathsf{Sm}\, V.$$

Type application is itself a program, so the typing rules become programs. The identity combinator I can then be its own type, $I : I$ because its type application mirrors evaluation:

$$I \,\#\, U \Longrightarrow \mathsf{Sm}\, U.$$

The polymorphism of the identity appears in how its type applies to different argument types. Programs such as I are *concrete*: they contain no *label*, the marker that tags a program with a name. Each concrete program is its own type, with itself as its only value. So a type with two or more values must be *abstract*. For example, the type Bool of booleans must satisfy

$$\mathsf{Bool} \,\#\, (A * B) \Longrightarrow \mathsf{Sm}\, A \quad \text{(only if } A \text{ equals } B\text{)}.$$

This condition ensures that the two arms of a conditional have the same type, so that the result has the same type whichever boolean is chosen, that is, whichever projection from the pair is taken. Traditional combinators cannot check this equality [21, Section 8.4], so it must be checked outside the system. Tree calculus can, using a program equal (Sections 2.3 and 5.1).

The abstract types are collectively described by a program called ontology that maps names to *abstractors*. Choosing it is a programming exercise, and the choice determines how informative typing is. Evaluation preserves typing whenever ontology is sound, that is, whenever its abstractors are. For example, the abstractor for booleans must introduce the equality check above.

When ontology is concrete, the type inference program infer is typed by evaluating its self-application, as in Figure 2. Here infer is applied to itself as it stands, without needing quotation to separate use from mention [30].

$$\mathsf{infer} \mathbin{@} \mathsf{infer} \Longrightarrow \mathsf{Sm}\, \mathsf{infer}$$

Figure 2. Self-application of the inference program

The tooling required for implementing the calculus may become typed programming. Examples include patching and measuring and optimising (Section 12).

The contributions of the paper are as follows.

- Types are trees; typing rules are programs; and type application displaces function types, type variables and quantifiers.
- Abstract types are declared by a program, the ontology, and typing is sound if the ontology is.
- One sound ontology supports general recursion, polymorphic lists, type-directed conditionals, length-indexed arrays, and generic and specialised queries and updates.
- A type inference program is its own type.
- Some tooling can be typed.

All theorems in the paper have been mechanised in Rocq [23]. Its README maps the sections of the paper to Rocq files, and each Theorem to its Rocq name. The paper presents the key ideas and arguments; the full details are in the formal proofs. Each Theorem bears the name of its Rocq counterpart. Some of the lemmas are also referenced here.

Section 1 is the introduction. Section 2 reprises tree calculus. Section 3 gives the typing rules as programs and proves them sound. Section 4 treats concrete types. Section 5 treats abstract types. Section 6 treats type-directed conditionals. Section 7 treats arrays. Section 8 treats queries. Section 9 treats updates. Section 10 treats function types. Section 11 combines all the abstract types built in this paper into one sound ontology. Section 12 considers typed tooling. Section 13 discusses related work. Section 14 concludes. Appendix A places the work in the context of the Leibniz project.

2 Tree Calculus

Typing must analyse types, so if types are programs then programs must analyse programs. Tree calculus does this natively, through triage, which inspects a program as it stands. This section presents the calculus and the intensional constructions that typing will use, including the traditional *combinators* [12, 33] as well as programs that analyse structure. For the broader picture, tree calculus and its reflective programs are developed in depth in [21], and typed program analysis using conventional typing is developed in [22].

2.1 Values, expressions and evaluation

Previously, tree calculus has been construed as a rewriting system [21], whose normal forms are finite binary trees. A term is *stable* if no rewriting step applies to it. Analysis of an unstable term is unreliable, because no program can

distinguish a term from its reduct. Here, expressions are represented by stable trees. Their evaluation is given a big-step operational semantics [29]. The relation $e \Longrightarrow v$ relates a stable expression e to a stable value v. In this way, the construction of an expression is separated from its evaluation. Type inference can then be programmed for expressions as well as values, and an update can patch a computation (Section 12.1), even while it is paused (Section 12.4).

The *values*, or *programs*, are the *leaves*, *stems* and *forks* of Figure 3. They correspond to the normal forms of the rewriting system. They are the targets of evaluation. Expressions use the same binary syntax to represent applications ready for evaluation. Under this reading, a stem $\mathsf{Sm}\, v$ marks the value v as an expression, while a fork $\mathsf{Fk}\, e_1\, e_2$ is the application of two expressions. Application of two values, and of an expression to a value, are written $v_1 \mathbin{@} v_2$ and $e \mathbin{@_r} v$ respectively (Figure 3). Both operators associate to the left. Programs and expressions are finite binary trees without variables, and so are closed. Every tree can be read as an expression, but not every expression can be evaluated; in particular, a leaf cannot be evaluated.

The evaluation rules use three forms of fork (Figure 3). In $K_1 u$ and $S_2[g, f]$ the subscripts record the number of arguments already stored. Central to program analysis is the third form, *triage*. In $\mathsf{triage}[f_0, f_1, f_2]$ the program f_0 is the *leaf case*, f_1 the *stem case* and f_2 the *fork case*.

The first rule says that a marked value $\mathsf{Sm}\, v$ evaluates to v; the next two evaluate the left and right sides of an application. The remaining rules concern applications $v_1 @ v_2$ of values. The first two build stems and forks. The next two supply traditional K- and S-computation. The final three let a program inspect its argument by triage: it returns the leaf case, applies the stem case to the child of a stem, or applies the fork case to the two children of a fork.

Evaluation is eager: both component expressions of an application must evaluate before the resulting operator is applied, even if $K_1 t$ will discard the argument.

Write $e_2 \rightsquigarrow e_1$ when $e_1 \Longrightarrow v$ implies $e_2 \Longrightarrow v$ for every value v; that is, to evaluate e_2 it suffices to evaluate e_1. Each rule of Figure 3 with exactly one premise gives such a step, as in $S_2[g, f] @ u \rightsquigarrow \mathsf{Fk}\, (g @ u)\, (f @ u)$. These steps correspond to the reduction steps of a rewriting system. However, the rules that evaluate the two sides of an application have two premises, so an evaluation is a tree of such steps rather than a sequence. Section 12.4 uses a small-step semantics, derived from this one in Rocq, to obtain a sequence.

Theorem 1 (`eval_deterministic`).
If $e \Longrightarrow v$ and $e \Longrightarrow v'$ then $v = v'$.

The traditional combinators are easily defined as values (Figure 4), as are the programs of the rest of this section. Some of these programs are only specified in the figure: equality; bz; the recognisers; and the concreteness test. Equality is

$$M ::= \mathsf{Lf} \mid \mathsf{Sm}\, M \mid \mathsf{Fk}\, M\, M$$

$$\begin{aligned} v_1 @ v_2 &= \mathsf{Fk}\, (\mathsf{Sm}\, v_1)\, (\mathsf{Sm}\, v_2) \\ e \mathbin{@_r} v &= \mathsf{Fk}\, e\, (\mathsf{Sm}\, v) \\ K_1 u &= \mathsf{Fk}\, \mathsf{Lf}\, u \\ S_2[g, f] &= \mathsf{Fk}\, (\mathsf{Sm}\, g)\, f \\ \mathsf{triage}[f_0, f_1, f_2] &= \mathsf{Fk}\, (\mathsf{Fk}\, f_0\, f_1)\, f_2 \end{aligned}$$

$$\frac{}{\mathsf{Sm}\, v \Longrightarrow v}$$

$$\frac{\mathsf{Fk}\, e_{11}\, e_{12} \Longrightarrow v_1 \qquad \mathsf{Fk}\, (\mathsf{Sm}\, v_1)\, e_2 \Longrightarrow v}{\mathsf{Fk}\, (\mathsf{Fk}\, e_{11}\, e_{12})\, e_2 \Longrightarrow v}$$

$$\frac{\mathsf{Fk}\, e_{21}\, e_{22} \Longrightarrow v_2 \qquad \mathsf{Fk}\, (\mathsf{Sm}\, v_1)\, (\mathsf{Sm}\, v_2) \Longrightarrow v}{\mathsf{Fk}\, (\mathsf{Sm}\, v_1)\, (\mathsf{Fk}\, e_{21}\, e_{22}) \Longrightarrow v}$$

$$\frac{}{\mathsf{Lf} @ u \Longrightarrow \mathsf{Sm}\, u}$$

$$\frac{}{(\mathsf{Sm}\, t) @ u \Longrightarrow \mathsf{Fk}\, t\, u}$$

$$\frac{}{K_1 t @ u \Longrightarrow t}$$

$$\frac{\mathsf{Fk}\, (g @ u)\, (f @ u) \Longrightarrow v}{S_2[g, f] @ u \Longrightarrow v}$$

$$\frac{}{\mathsf{triage}[f_0, f_1, f_2] @ \mathsf{Lf} \Longrightarrow f_0}$$

$$\frac{f_1 @ u \Longrightarrow v}{\mathsf{triage}[f_0, f_1, f_2] @ (\mathsf{Sm}\, u) \Longrightarrow v}$$

$$\frac{f_2 @ u_1 \mathbin{@_r} u_2 \Longrightarrow v}{\mathsf{triage}[f_0, f_1, f_2] @ (\mathsf{Fk}\, u_1\, u_2) \Longrightarrow v}$$

Figure 3. Evaluation

semantically equivalent to that of [21]. All of them are formally defined in the Rocq development. The value $g \circ f$ represents composition: to evaluate $(g \circ f) @ x$ is first to evaluate $f @ x$ and then to apply g to the resulting value. A swap supplies a stored value to its argument: evaluation of $\mathsf{swap}[p] @ f$ becomes evaluation of $f @ p$. Arrays (Section 7) and wrapped types (Section 5.6) use swaps to supply their entries and payloads.

2.2 Pairs and projections

A pair supplies its two components to a consumer. The projections choose one component using the combinatory selectors K and KI (Figure 4). Evaluation of $(u * v) @ f$ becomes evaluation of $f @ u \mathbin{@_r} v$ (`pair_apply`), so that $\mathsf{fst} @ (u * v) \Longrightarrow u$ and $\mathsf{snd} @ (u * v) \Longrightarrow v$.

$$
\begin{array}{rcl}
K & = & \mathsf{Sm}\,\mathsf{Lf} \\
I & = & S_2[K,K] \\
KI & = & K_1 I \\
g \circ f & = & S_2[K_1 g, f] \\
\mathsf{swap}[p] & = & S_2[I, K_1 p] \\
u * v & = & S_2[\mathsf{swap}[u], K_1 v] \\
\mathsf{fst} & = & \mathsf{swap}[K] \\
\mathsf{snd} & = & \mathsf{swap}[KI] \\
\mathsf{isLeaf} & = & \mathsf{triage}[K, K_1 KI, K_1(K_1 KI)] \\
\mathsf{equal}\ @\ t\ @_r\ t & \Longrightarrow & K \\
\mathsf{equal}\ @\ t\ @_r\ u & \Longrightarrow & KI \quad (t \neq u) \\
\mathsf{wait}[M,N] & = & S_2[S_2[K_1 M, K_1 N], I] \\
\mathsf{wait2}[M,N,x] & = & S_2[S_2[S_2[K_1 M, K_1 N], K_1 x], I] \\
\mathsf{dup_z} & = & S_2[I,I] \\
\mathsf{bz}\ @\ g & \Longrightarrow & \mathsf{wait}[\mathsf{dup_z}, g] \\
Zf & = & \mathsf{wait}[\mathsf{dup_z}, f \circ \mathsf{bz}] \\
\mathsf{decorate}[f,u] & = & \mathsf{wait}[K_1 f, u] \\
\mathsf{label}[n] & = & S_2[K, K_1 n] \\
\mathsf{tagged}[f,n] & = & f \circ \mathsf{label}[n] \\
\mathsf{tagger}[n] & = & S_2[\mathsf{Lf} \circ (\mathsf{Lf} \circ K), \\
 & & \quad ((\mathsf{Sm}(\mathsf{Sm}\,K)) \circ K) \circ K_1 n] \\
\mathsf{get_tagname}\ @\ \mathsf{tagged}[f,n] & & \\
 & \Longrightarrow & \mathsf{Sm}\,n \\
\mathsf{get_tagname}\ @\ u & \Longrightarrow & \mathsf{Lf} \quad (u \text{ not tagged}) \\
\mathsf{get_tagged}\ @\ \mathsf{tagged}[f,n] & & \\
 & \Longrightarrow & \mathsf{Sm}\,(\mathsf{Fk}\,f\,n) \\
\mathsf{get_tagged}\ @\ u & \Longrightarrow & \mathsf{Lf} \quad (u \text{ not tagged}) \\
\mathsf{name_test}[n] & = & \mathsf{wait}[\mathsf{equal}, \mathsf{Sm}\,n] \circ \mathsf{get_tagname} \\
\mathsf{tagged_by}[n] & = & K_1(\mathsf{Lf} \circ \mathsf{name_test}[n]) \\
\mathsf{is_concrete}\ @\ v & \Longrightarrow & K \quad (v \text{ concrete}) \\
\mathsf{is_concrete}\ @\ v & \Longrightarrow & KI \quad (\text{otherwise})
\end{array}
$$

Figure 4. Programs of tree calculus, defined by = or specified by $\Longrightarrow$

2.3 Intensionality

Triage lets a program inspect the construction of its argument. The simplest test returning K or KI is isLeaf (Figure 4) which returns K on a leaf and KI on a stem or a fork. Equality, which compares trees recursively, supplies the comparison of types that typing requires. It satisfies the equations of Figure 4 (`equal_eq`, `equal_neq`). The partial application equal @ t evaluates to the test for equality with t (`equal_eval`). Although K and KI are enough to discriminate in an untyped setting, the typed equality of Section 8.2 will return booleans, as defined in Section 5.1.

2.4 Control, recursion and decoration

Control requires choosing when an application is evaluated, since an application supplied as an argument is evaluated before the receiving program can use it. To postpone that computation, it must be stored in a value until a later application releases it. The construction *waiting* wait$[M,N]$ does this (Figure 4). Constructing this value stores M and N without applying one to the other. Then wait$[M,N]$ @ $P \rightsquigarrow M$ @ N @$_r$ P. The further argument P thus releases the waiting application. In the same way, wait2$[M,N,x]$ stores three programs. Then wait2$[M,N,x]$ @ $P \rightsquigarrow M$ @ N @$_r$ x @$_r$ P. Recursion and decoration illustrate two uses of this control.

Waiting postpones a self-application to yield a *fixfunction* Zf with *body* f. The definition of Z is in Figure 4. Evaluation of Zf @ x becomes evaluation of f @ (Zf) @$_r$ x. The two evaluations have the same results (`Z_unfold`, `Z_unfold_inv`). In this manner, unlike the traditional fixpoints, Zf is stable: it is a program, even when its application to x diverges. Waiting thereby separates the finite representation of recursion from the potentially unbounded evaluation.

A *decoration* stores a program f together with a value u (Figure 4). Evaluation of decorate$[f,u]$ @ x becomes evaluation of f @ x. Before application, the stored value u is available to intensional analysis.

2.5 Tagging

A program is tagged with a *nametag* n by composing it with the *label* of n. The label is an identity that carries n (Figure 4). The program f is the *carrier* of tagged$[f,n]$. Evaluation of tagged$[f,n]$ @ u becomes evaluation of f @ u. Since the label occupies a fixed position, structural analysis can still recognise a tagged value and recover both f and n. Like decoration, tagging is a construction from existing programs. Tagging will be central to the construction of abstract types and their values.

A nametag may be any program. The nametags used for types are concrete *names*, such as bool_nm for the booleans. Most names are *indices* $\mathsf{Sm}^k\,\mathsf{Lf}$. The name fun_nm of the optional function types (Section 10) is the leaf, the least index. The name list_nm is $K_1\mathsf{Lf}$ to keep the tag and the test of a list small. The particular trees are otherwise immaterial.

The recognisers get_tagname and get_tagged recover the nametag, or the carrier and the nametag, of a tagged value, and return Lf on an untagged one (Figure 4). Both analyse their argument by pattern matching, and so by triage (Section 2.7).

A *name test* name_test$[n]$ compares the nametag of a value with n. It returns K on any value tagged with n and KI otherwise. It examines only the nametag. The program tagged_by$[n]$ ignores its first argument, runs the name test on its second and marks the answer as a success, so that tagged_by$[n]$ @ a @$_r$ $U \Longrightarrow \mathsf{Sm}\,b$ when name_test$[n]$ @ $U \Longrightarrow b$ (`tagged_by_eval`).

2.6 Concrete and abstract programs

Tagging divides programs into three classes. A program is *abstract* when it is tagged at the root, that is, of the form tagged$[f,n]$ whatever f may be. It is *concrete* when none of its subtrees is a label, so that no construction from concrete

parts can form a tag. Many programs are neither, such as a pair of abstract programs.

The program is_concrete decides this: is_concrete @ v evaluates to K when v is concrete, and to KI otherwise (`is_concrete_spec`). A concrete program contains no tagged subtree (`concrete_tag_free`), but may contain arbitrary label-free program structure, including triages, which may even compute labels and abstract programs at run time. For example, the *tagger* tagger$[n]$ (Figure 4) builds the label of n only when it is applied, so that tagger$[n]$ @ $x \Longrightarrow$ tagged$[x, n]$. It is concrete whenever n is (`tagger_concrete`). This is what will allow the inference program, which builds abstract types, to be concrete itself (Section 4).

2.7 Pattern matching

The intensionality of tree calculus supports pattern-matching functions. In such functions, pattern variables produce abstractions that the optimised bracket abstraction star compiles. Triage [21] and equality tests handle other cases. A pattern may be a constant such as fst or contain pattern variables, as in tagged$[E, N]$ and $S_2[G, H]$ where E, N, G and H are pattern variables. Pattern variables, and the λ-abstractions such as $\lambda r.\ \mathsf{neg} \circ r$ of later sections, are the only concession to variable binding in the set-up. Both are compiled away by star. So programs and expressions hold no variables, and the same trees could be written out with S_2, K_1 and equal. The wildcard _ matches anything. A variable repeated in a pattern stands for a test that the subtrees are equal. A clause is of the form

$$\mid p \Rightarrow e$$

where p is a pattern and e is an expression. Here $\Rightarrow$ is a short arrow, not to be confused with the long arrow $\Longrightarrow$ of evaluation.

A definition given only by clauses matches on its unnamed argument, as in infer =| Lf $\Rightarrow$ Sm Lf | .... A definition may also name further arguments. For example, the clauses of type_app @ F @$_r$ U match on F. Their bodies are functions of U. Two further forms are used:

- **match** e **with** ... **end** analyses an intermediate result by clauses. Its **end** marks the scope;
- **let** Sm $X = e$ **in** e' evaluates e; if the result is Sm X it continues with e' and otherwise the whole result is the failure Lf.

Conditionals are programs too. A test such as equal returns one of the selectors K and KI. The conditional applies this selector to two arms. Then K chooses the first arm and KI the second. Evaluation is eager, so both arms are evaluated before the choice, unless they are functions still awaiting an argument.

```
type_app @ F @_r U = match F with
| Lf ⇒ Sm (Sm U)
| Sm G ⇒
    match Fk G U with
    | tagged[C, N] ⇒
        let Sm E = wait2[ontology, N, type_app] @ C
        in Sm tagged[E, N]
    | X ⇒ Sm X
    end
| tagged[E, N] ⇒ wait[E, type_app] @ U
| Fk (tagged[E, N]) F_2 ⇒ Lf
| triage[F_0, F_1, F_2] ⇒
    match U with
    | tagged[C, M] ⇒ Lf
    | Lf ⇒ Sm F_0
    | Sm X ⇒ wait[type_app, F_1] @ X
    | Fk X Y ⇒ let Sm G = F_2 # X in G # Y
    end
| K_1 G ⇒ Sm G
| S_2[G, H] ⇒
    let Sm V = G # U in
    let Sm W = H # U in V # W
end

F # U = type_app @ F @_r U

infer =
| Lf ⇒ Sm Lf
| Sm u ⇒ let Sm U = infer @ u in Sm (Sm U)
| Fk u_1 u_2 ⇒
    let Sm U_1 = infer @ u_1 in
    let Sm U_2 = infer @ u_2 in (Sm U_1) # U_2

infer_expression =
| Lf ⇒ Lf
| Sm u ⇒ infer @ u
| Fk e_1 e_2 ⇒
    let Sm T_1 = infer_expression @ e_1 in
    let Sm T_2 = infer_expression @ e_2 in T_1 # T_2
```

Figure 5. Typing rules as programs. The concrete clauses of type application mirror evaluation, and its abstract clauses forget how values are built. The figure gives type application and inference, using implicit ontology.

3 Typing Rules as Programs

The goal of this section is to make typing rules into programs (Figure 5). Traditionally, typing rules derive typing judgements of the form $\Gamma \vdash t : T$ where Γ is a context, t is a term and T is a type. These derivations are computations that live outside of the term language, since the terms cannot analyse one another. Tree calculus internalises this computation as follows.

1. The derivation relation is replaced by a type inference function, so that each program has at most one type.

2. The inference rules become clauses of pattern-matching programs.
3. Programs are closed, so inference acts on the program alone, without a context.
4. The typing judgement $t : T$ becomes the evaluation infer @ $t \Longrightarrow \mathsf{Sm}\, T$ where $\mathsf{Sm}\, T$ indicates success and Lf indicates failure. Thus types are programs, and expressions, being trees, are typed by a program too.
5. The program for type inference becomes the definition, and the old rules become theorems.

Two things make this harder than transcribing the rules. First, tree calculus is combinatory, so there are no rules for variables and abstraction to share the work, and all of it falls on type application. Second, if type application only mirrors evaluation then programs are their own types, as the concrete programs are (Section 4), whereas the truth values, for example, must share a type. So type application has two kinds of clause: *concrete* clauses, which mirror evaluation, and *abstract* clauses, which forget how values are built. The abstract types are introduced through an implicit parameter ontology. Applied to the name of an abstract type and then to type application, it returns an *abstractor* for that name (Section 3.2). Typing is then sound when ontology is, in one of two senses: by typing (Section 3.3) or by values (Section 3.4).

3.1 Type inference

Write

$$\begin{array}{lll} F \,\#\, U & \text{for} & \text{type_app} \;@\; F \;@_r\; U \\ u : U & \text{for} & \text{infer} \;@\; u \Longrightarrow \mathsf{Sm}\, U \\ e :: U & \text{for} & \text{infer_expression} \;@\; e \Longrightarrow \mathsf{Sm}\, U. \end{array}$$

Type application is chained to the left: $F \,\#\, U \,\#\, V \Longrightarrow \mathsf{Sm}\, W$ means that $F \,\#\, U \Longrightarrow \mathsf{Sm}\, G$ and $G \,\#\, V \Longrightarrow \mathsf{Sm}\, W$ for some G. A program v is *self-typed* when $v : v$. An expression is itself a tree, so it may have a *value type* T with $e : T$ as well as an *expression type* U with $e :: U$. Here Lf and Sm play two roles: they form programs and expressions; and they record the failure and success of type application and inference. The program infer_expression at the bottom of Figure 5 types expressions. A stem $\mathsf{Sm}\, u$ represents the value u and is typed by infer. An application is typed by type application, and a bare leaf, which cannot be evaluated, fails. This gives the derived rule

Theorem 2 (`infer_application`).
If $f : F$ and $u : U$ and $F \,\#\, U \Longrightarrow \mathsf{Sm}\, V$ then $f \;@\; u :: V$.

The program infer above it gives a leaf the type Lf and types a stem $\mathsf{Sm}\, u$ by $\mathsf{Sm}\, U$ where $u : U$. It types a fork $\mathsf{Fk}\, u_1\, u_2$ by $(\mathsf{Sm}\, U_1) \,\#\, U_2$ where $u_1 : U_1$ and $u_2 : U_2$. So it all comes down to type application.

3.2 Type application

The program type_app for type application is a recursive, pattern-matching function with clauses for leaves, stems and various forks. Most clauses are concrete, in that they reflect the evaluation rules in Figure 3. Consequently, they give each concrete program of Section 2.6 itself as its type (Section 4). The exceptions all involve the tagged types, which are the abstract types. If a triage type is applied to a tagged argument type, then type application fails. A fork type whose left branch is tagged also fails, since a tagged type is itself a fork, and otherwise the triage clause would match it. This refusal ensures that nothing relies on what an abstract type has forgotten. If the type being applied is a tagged type $\mathsf{tagged}[E, N]$ then E is its *eliminator*. Type application evaluates E @ type_app $@_r\; U$ for each argument type U. When the stem clause would build a tagged type $\mathsf{tagged}[C, N]$ it builds $\mathsf{tagged}[E, N]$ instead, where

$$\text{ontology} \;@\; N \;@_r\; \text{type_app} \;@_r\; C \Longrightarrow \mathsf{Sm}\, E.$$

The value $\mathcal{A}$ of ontology @ N $@_r$ type_app is the *abstractor* at N. It maps the carrier type C to $\mathsf{Sm}\, E$ or refuses C by returning Lf. The stem clause waits on ontology so that ontology is applied only when an abstract type is built.

Function types (Section 10) are the one family assumed throughout, since they can give a program built from the other families its traditional type, as for minimisation_fn : (Nat → Bool) → (Nat → Nat) (Theorem 47). So ontology @ fun_nm ⟹ function_abstractor unless stated otherwise. In the terms of Section 5.1, ontology supports function types. The Theorems state any other family that they need.

Being programs, type application and inference need not terminate. Since the concrete clauses mirror evaluation, the typing of a program can diverge as its evaluation does. For example, let ω = dup_z = $S_2[I, I]$. The expression $\omega \;@\; \omega$ has no type, because typing it computes $\omega \# \omega$ and this diverges as $\omega \;@\; \omega$ does (`infer_expression_omega_omega_no_value`). So inference is a semi-decision procedure in general. Where a recursive program has an *invariant*, a type for its recursive call that typing can use without unfolding the recursion, typing stops there, whether or not evaluation does (Sections 8.3 and 9.4). Conditions that guarantee termination in general are left to future work (Appendix A).

3.3 Soundness by typing

Typing is *sound* for a given ontology if *evaluation preserves typing* (EPT). That is, if $e :: T$ and $e \Longrightarrow v$ then $v : T$. Soundness here is preservation alone, since progress comes from the rules of evaluation. Every application of two values matches a rule of Figure 3. So evaluation can stop short of a value only at a bare leaf that stands as an expression. An expression with an expression type holds no such leaf (`expression_typed_leaf_free`). So evaluation of a typed expression either yields a value or runs forever (typed_

progress). Since the concrete clauses of type application mirror evaluation, soundness comes down to the abstract clauses of Figure 5.

The stem clause replaces a carrier type C by an eliminator E where the abstractor $\mathcal{A}$ at the nametag gives $\mathcal{A}\ @\ C \Longrightarrow \mathsf{Sm}\ E$ (Section 3.2). Soundness by typing asks that C apply as E does. An abstractor $\mathcal{A}$ is *sound by typing* if $\mathcal{A}@C \Longrightarrow \mathsf{Sm}\ E$ and $E\ @\ \text{type_app}\ @_r\ U \Longrightarrow \mathsf{Sm}\ V$ imply $C\ \#\ U \Longrightarrow \mathsf{Sm}\ V$ for all C, E, U, V. Then ontology is *sound by typing* if every abstractor that it returns, at every nametag, is sound by typing (Figure 6).

Theorem 3 (ept_by_typing).
If ontology is sound by typing then typing is sound.

The proof is in Rocq, by induction on the evaluation $e \Longrightarrow v$: each rule of Figure 3 preserves the type of the expression. The one interesting case is the application of a tagged value to a value of type U. Its carrier has some type C and since the abstractor at its nametag is sound by typing, $C\ \#\ U$ gives the required type.

3.4 Soundness by values

Soundness by typing requires $C\ \#\ U$ to succeed, but type application refuses a triage at an abstract argument type (Figure 5). So an abstractor that accepts a triage as a carrier type need not be sound by typing, although each carrier of that type may still give its result the type that the eliminator promises. A program can read the structure of a value whose type has forgotten it. A weaker condition covers such abstractors. Type-directed conditionals are the first to need it (Section 6).

The general definition is about values, with proofs by induction on a bound on their evaluation.

Write $e \Longrightarrow_n v$ when e evaluates to v by a derivation of depth at most n. A bound can always be raised: if $e \Longrightarrow_m v$ and $m \le n$ then $e \Longrightarrow_n v$ (eval_bounded_le).

Theorem 4 (eval_iff_bounded).
$e \Longrightarrow v$ if and only if $e \Longrightarrow_n v$ for some n.

Evaluation preserves typing at bound n (EPT(n)) if $e :: T$ and $e \Longrightarrow_n v$ imply $v : T$.

An abstractor $\mathcal{A}$ is *sound by values* if the conditions of Figure 6 imply $v : V$ for all C, E, U, V, c, u, v, n. That is, a carrier c applied to a value u within bound n gives a value of the type that E promises, and the proof may assume EPT(n) at that bound. Then ontology is *sound by values* if every abstractor that it returns is sound by values.

An abstractor that is sound by typing is sound by values, and so is an ontology that is sound by typing. Soundness by typing gives $C\ \#\ U \Longrightarrow \mathsf{Sm}\ V$. So $c\ @\ u :: V$ and then EPT(n) gives $v : V$.

Theorem 5 (evaluation_preserves_typing, EPT).
If ontology is sound by values then typing is sound.

Evaluation preserves typing (EPT)
$e :: T$ and $e \Longrightarrow v$ imply $v : T$

Abstractor $\mathcal{A}$ sound by typing
$\mathcal{A}\ @\ C \Longrightarrow \mathsf{Sm}\ E$ and $E\ @\ \text{type_app}\ @_r\ U \Longrightarrow \mathsf{Sm}\ V$ imply $C\ \#\ U \Longrightarrow \mathsf{Sm}\ V$

Abstractor $\mathcal{A}$ sound by values
$\mathcal{A}\ @\ C \Longrightarrow \mathsf{Sm}\ E$ and $c : C$ and $E\ @\ \text{type_app}\ @_r\ U \Longrightarrow \mathsf{Sm}\ V$ and $u : U$ and EPT(n) and $c\ @\ u \Longrightarrow_n v$ imply $v : V$

Sound ontology
ontology is sound by typing (by values) if every abstractor $\mathcal{A}$ with ontology $@\ N\ @_r\ \text{type_app} \Longrightarrow \mathcal{A}$ is sound by typing (by values)

Figure 6. Soundness

c concrete $\Longrightarrow$ $c : c$ and c is the only value of type c
t closed and β-normal $\Longrightarrow$ $[[t]]$ concrete
f concrete $\Longrightarrow$ Zf concrete
ontology concrete $\Longrightarrow$ infer : infer
The concrete programs form a self-typing, Turing-complete class.

Figure 7. Self-typing. Here $[[t]]$ is the translation of a pure lambda term t into tree calculus.

This is the central theorem. The proof is in Rocq. It shows EPT(n) by induction on n and then deduces EPT by Theorem 4. A tagged value tagged$[c, m]$ applies to a value u by evaluating $c\ @\ u$ at a smaller bound than the whole. So the induction hypothesis supplies the premise EPT(n) of soundness by values for the abstractor at m. That soundness then gives the result its type.

4 Concrete Types

How far can typing go if it forgets nothing? Typing a concrete program uses only the concrete clauses, so the program is self-typed whatever ontology may be. Even so, such types go a long way. They already give polymorphism (Section 4.1), and the concrete programs form a Turing-complete class that includes the inference program itself (Section 4.2). The obstacle is recursion: a type that records everything records the recursion too, so typing a recursion can run it (Section 4.3). Stopping it requires types that forget, which Section 5 introduces.

4.1 Self-typing

Theorem 6 (concrete_is_self_typed).
If c is concrete then $c : c$.

Theorem 7 (concrete_type_unique_value).
If $v : T$ and T is concrete then $v = T$.

More generally, a *tag-free* program, one with no tagged subtree, is self-typed and the only value of its type (`infer_tag_free`, `infer_tag_free_inv`). Every concrete program is tag-free (Section 2.6). Concreteness is the stronger condition: it is what the constructions of Section 4.2 preserve, and what is_concrete tests.

A concrete type has only one value, yet concrete types already support polymorphism. Traditionally, the identity has the type $\forall X.\, X \to X$. Here the identity $I = S_2[K, K]$ is concrete, so $I : I$ and $I \# U \implies \mathsf{Sm}\, U$ for every value U (Section 3.2).

Theorem 8 (`identity_polymorphic`).
If $u : U$ then $I \mathbin{@} u :: U$.

Similarly, K is traditionally typed by the instances $U \to V \to U$ or by $\forall X.\forall Y.\, X \to Y \to X$ [15, 18]. Here $K : K$. If $u : U$ then $K_1 u : K_1 U$ and $K_1 U \# V \implies \mathsf{Sm}\, U$ for every V. The type $K_1 U$ mentions neither $V \to U$ nor any quantified type.

4.2 Which programs are concrete

Let $[\![t]\!]$ be the translation of a closed pure lambda term t into tree calculus by the optimised translation star of lambda abstraction.

Theorem 9 (`normal_combinators_are_concrete`).
If t is a closed pure lambda term in β-normal form then $[\![t]\!]$ is concrete.

Normality ensures that the translation does not produce a label. The usual fixpoint combinators are not normal, but recursion can instead be defined by the fixfunction constructor Z and this preserves concreteness.

Theorem 10 (`Z_preserves_concrete`).
If f is concrete then so is Zf.

The μ-recursive programs are built from concrete components, with Z for recursion, so the concrete programs form a Turing-complete class.

Theorem 11 (`concrete_is_Turing_complete`).
Let f be a μ-recursive function of positive arity acting on the indices $\mathsf{Sm}^k\,\mathsf{Lf}$ of Section 2.5. Then f is computed by a concrete program P in the sense that if f maps arguments $\vec{x}$ to y then applying P to the indices of $\vec{x}$ evaluates to the index of y.

Being concrete, such a program is self-typed. Hence, different programs have different types, which is not very informative. Once a type Nat of natural numbers is defined in Section 5.2, type application to the k-fold product Nat^k can reveal their shared nature in Section 9.4.

Concrete programs can also be intensional, since triage does not introduce labels. In particular, the typing programs are concrete whenever ontology is (`infer_concrete` and `type_app_concrete`). Although they build abstract types, they contain no labels, since they build each label at run time from its nametag.

Theorem 12 (`infer_self_typed`).
If ontology is concrete then infer : infer.

4.3 Concrete recursion

Recursive programs can be concrete, and so typed, but their very concreteness means that non-termination of their evaluation can become non-termination of their typing. For example, consider a concrete program for dithering. The negation cneg applies its argument to the selectors KI and then K. The program cdither $= Z(\lambda r.\, \mathsf{cneg} \circ r)$ negates its own result. It is concrete, so cdither : cdither (`cdither_concrete`), but cdither $@\, x$ has no value for any x (`cdither_no_value`). When x has a type, the typing of the application does not finish either.

Theorem 13 (`infer_expression_cdither_no_value`).
If x has a type then infer_expression @ (cdither $@\, x$) has no value.

If typing of such recursive programs is to terminate then typing must forget some information, so that invariants (Section 3.2) can emerge. The abstract types of Section 5 show how to forget. Then query and update types support invariants (Sections 8 and 9).

5 Abstract Types

Concrete programs are self-typed, whereas some values must share a type, as the truth values must. The solution is to introduce abstract types, which forget how their values were built. The tagged values are re-purposed as abstract values. Two obstacles shape the families of this section. First, elimination must use only what introduction records, so each family comes with a soundness condition on its carrier types, the last of the four parts of its figure. Second, a value with no entry, such as an empty list, gives no entry from which to read its type, so it holds a ghost, a value that stands in for its type (Section 5.3).

5.1 Booleans

The booleans show how any abstract type is built. In the untyped tree calculus, the values K and KI can be used as truth values, with their next two arguments supplying the arms of a conditional. For typing, it is more convenient to collect the arms as a pair, so that a boolean acts as a projection fst or snd and the types of the two arms form a single argument type. However, these projections are concrete, so each is self-typed, whereas the booleans must share a type. So they must be made abstract.

The first step is to reserve a *name* bool_nm for the booleans, which tags both the values and their type. A name is concrete, and so self-typed. The abstract types with the same name form a *family*, such as the lists of each element

Type specification
Bool # $U \Longrightarrow$ Sm V iff $U = V * V$
tt : Bool
ff : Bool

Type elimination
bool_eliminator @ type_app0 =
| $U_1 * U_2 \Rightarrow$ **if** equal @ U_1 @$_r$ U_2 **then** Sm U_1 **else** Lf
| _ $\Rightarrow$ Lf
Bool = tagged[bool_eliminator, bool_nm]

Type introduction
tt = tagged[fst, bool_nm]
ff = tagged[snd, bool_nm]
bool_abstractor =
| fst $\Rightarrow$ Sm bool_eliminator
| snd $\Rightarrow$ Sm bool_eliminator
| _ $\Rightarrow$ Lf

Soundness
fst # $(V * V) \Longrightarrow$ Sm V
snd # $(V * V) \Longrightarrow$ Sm V

Figure 8. The booleans

type, with their name list_nm. Now the truth values can be defined as

$$\begin{array}{lcl} \mathsf{tt} & = & \mathsf{tagged}[\mathsf{fst}, \mathsf{bool_nm}] \\ \mathsf{ff} & = & \mathsf{tagged}[\mathsf{snd}, \mathsf{bool_nm}]. \end{array}$$

Each constructor tags a *carrier* with the name, as tt tags fst. The type of a carrier is its *carrier type*, so here the projections are both carriers and carrier types.

That booleans are applied to pairs of arms that have the same type is captured by Bool # $(V * V) \Longrightarrow$ Sm V with failure otherwise. This is the elimination rule of the *type specification*, which says how values of the type are consumed, including the argument types at which application fails. Those failures ensure that nothing relies on what the type has forgotten. The introduction rules of the type specification give the types of the constructors, here tt : Bool and ff : Bool.

By the elimination clause of Figure 5, the elimination rule requires an eliminator bool_eliminator with bool_eliminator @ type_app @$_r$ $(V * V) \Longrightarrow$ Sm V. Tagging it with the name bool_nm gives the type Bool (Figure 8). Any parameters of a family, such as the element type of a list, are held in the eliminator of each type. In figures, type_app0 is the parameter through which an eliminator or abstractor receives type application. Figures give eliminators and abstractors by their clauses, which the Rocq programs satisfy (e.g. `bool_eliminator_some`, `bool_eliminator_inv` and `bool_eliminator_none`).

Under the concrete clauses alone, the truth values would be self-typed. Their types are determined instead by the abstractor at bool_nm (Section 3.2), which maps exactly the carrier types of the family to eliminators. Here the abstractor bool_abstractor maps both projections to Sm bool_eliminator. An abstractor $\mathcal{A}$ *abstracts* C to E when $\mathcal{A}$ @ $C \Longrightarrow$ Sm E. The carrier types that an abstractor abstracts to the same eliminator share one type, which is how an abstract type forgets its carrier.

ontology maps each name and then type application to the abstractor of its family, and any other nametag and type application to the *refusal* K_1 Lf that abstracts nothing. It *supports* booleans when ontology @ bool_nm $\Longrightarrow K_1$ bool_abstractor. Here K_1 discards the type application, which bool_abstractor does not need. In general, it supports a family when it takes the name of the family and type application to the abstractor of the family. A claim about a family assumes that ontology supports it.

Abstract types are introduced by a derived rule (`infer_tagged`). If the abstractor at N abstracts C to E then

$$\frac{c : C \qquad n : N}{\mathsf{tagged}[c, n] : \mathsf{tagged}[E, N]}.$$

For example, if ontology supports booleans then tt : Bool since fst and bool_nm are self-typed and the abstractor of the booleans abstracts fst to bool_eliminator. If $u : V$ and $v : V$ then $u * v : V * V$ and so tt @ $(u * v) :: V$ (Theorem 2). Since a name is self-typed, a value tagged with a name n has a type tagged with n. Conversely, a tagged program can only have an abstract type, whose nametag is the type of the program's nametag (`abstract_has_abstract_type`).

Soundness by typing (Section 3.3) is a condition on each abstractor. The abstractor of the booleans is sound by typing, since fst # $(V * V) \Longrightarrow$ Sm V and snd # $(V * V) \Longrightarrow$ Sm V. The refusal is sound by typing trivially.

Negation neg = swap[ff * tt] swaps tt and ff. Its type is Neg = swap[Bool * Bool] and Neg # Bool $\Longrightarrow$ Sm Bool.

Figure 8 has four parts: the type specification; the type elimination, which gives the eliminator; the type introduction, which gives the constructors and the abstractor; and soundness, which gives the condition at each carrier type. Each family of this section is described in these four parts.

5.2 Natural numbers

Natural numbers are the first recursive data type here. A number is a *fold*, which takes a pair of a base and a step. Zero gives the base, and the successor of n applies the step to the result of n. The recursion is in the data. So typing checks one step: a step f of type F must satisfy F # $U \Longrightarrow$ Sm U where U is the type of the base (Figure 9). This is a recursive call of type application, which the eliminator makes with the type application that it receives. By contrast, a Scott numeral passes its predecessor to the step, which leaves the recursion in the program.

A numeral k denotes the natural number succ^k[zero] except in the length of an array type, where it denotes an index (Section 7.1).

Type specification
Nat # $X \Longrightarrow \mathsf{Sm}\,U$ iff $X = U * F$ and $F \,\#\, U \Longrightarrow \mathsf{Sm}\,U$
zero : Nat
succ[n] : Nat if n : Nat

Type elimination
nat_eliminator @ type_app0 =
| $U * F \Rightarrow$ **let** Sm V = type_app0 @ F @$_r$ U **in**
if equal @ V @$_r$ U **then** Sm U **else** Lf
| _ ⇒ Lf
Nat = tagged[nat_eliminator, nat_nm]

Type introduction
zero = tagged[fst, nat_nm]
succ[n] = tagged[S_2[snd, n], nat_nm]
nat_abstractor =
| fst ⇒ Sm nat_eliminator
| S_2[snd, Nat] ⇒ Sm nat_eliminator
| _ ⇒ Lf

Soundness
fst # $(U * F) \Longrightarrow \mathsf{Sm}\,U$
S_2[snd, Nat] # $(U * F) \Longrightarrow \mathsf{Sm}\,U$ $\quad (F \,\#\, U \Longrightarrow \mathsf{Sm}\,U)$

Figure 9. Natural numbers

Type specification
Option[A] # $X \Longrightarrow \mathsf{Sm}\,U$ iff $X = U * F$ and $F \,\#\, A \Longrightarrow \mathsf{Sm}\,U$
none[a] : Option[A] if $a : A$
some[x] : Option[A] if $x : A$

Type elimination
option_eliminator[A] @ type_app0 =
| $U * F \Rightarrow$ **let** Sm V = type_app0 @ F @$_r$ A **in**
if equal @ V @$_r$ U **then** Sm U **else** Lf
| _ ⇒ Lf
Option[A] = tagged[option_eliminator[A], opt_nm]

Type introduction
none[a] = tagged[decorate[fst, a], opt_nm]
some[x] = tagged[S_2[snd, $K_1 x$], opt_nm]
option_abstractor =
| decorate[fst, A] ⇒ Sm option_eliminator[A]
| S_2[snd, $K_1 A$] ⇒ Sm option_eliminator[A]
| _ ⇒ Lf

Soundness
decorate[fst, A] # $(U * F) \Longrightarrow \mathsf{Sm}\,U$
S_2[snd, $K_1 A$] # $(U * F) \Longrightarrow \mathsf{Sm}\,U$ $\quad (F \,\#\, A \Longrightarrow \mathsf{Sm}\,U)$

Figure 10. Options

A fold carries its own recursion, so the types of numbers suffice to type programs on numbers. Addition acts on a pair. The program add receives $m * n$ and folds m with base n and a step succ_step that takes r to succ[r]. Multiplication folds m with base zero and a step that adds n to its result. Each is tag-free, and so is self-typed.

Theorem 14 (add_typed).
If ontology supports numbers then add : add and add # (Nat ∗ Nat) ⟹ Sm Nat.

Theorem 15 (mul_typed).
If ontology supports numbers then mul : mul and mul # (Nat ∗ Nat) ⟹ Sm Nat.

A number applies its step to the fold of its predecessor, so it does not hand over the predecessor itself. The program nat_pred recovers it by a fold over pairs, whose step takes $m*n$ to $n*$succ[n] and whose first component is the predecessor at the end. So it walks the whole of its argument (nat_pred_numeral).

Consider minimisation, a search over numbers: test a predicate p at a number n and stop if it holds, or otherwise continue from succ[n]. It can be written directly with Z on numbers. Evaluation is eager, so the recursive call waits on one more argument, which releases it only when it is chosen. This program runs (plain_search_runs). Yet for an ontology of booleans and numbers, type application refuses it at the type K_1Bool of the constant predicate K_1tt (plain_search_refused). Its conditional needs two arms of one type (Figure 8), whereas the arm that stops and the arm that continues have different types, since the second still holds the recursion. It will be typed as an update (Section 9.4).

Forgetting alone does not stop a recursion either. With the negation of Section 5.1, zdither = $Z(\lambda r.\,\mathrm{neg} \circ r)$ negates its own result. If ontology supports booleans then zdither has the type zdither_type = $Z(\lambda r.\,\mathrm{Neg} \circ r)$ (zdither_typed), and that type is as recursive as the program.

Theorem 16 (type_app_zdither_no_value).
For every U the application zdither_type # U has no value.

The same recursion, written as a query, is typed (Section 8.3).

5.3 Options

Traditionally, the option none has many types, of the form Option[A] but here inference is a program, so every program has at most one type, and none must somehow carry type information. Since the type A is also a program, it is tempting to pass A itself to none as in System F. But a type need not itself have a type, and if A has none then neither has none[A]. So none[a] takes a *ghost*, a value $a : A$ that stands in for its type, while some[x] needs no ghost (Figure 10). This has two implications. First, an option type is *inhabited*, that is, the type of some value, only if its type parameter is (option_is_data). Second, generic queries and updates must take extra care with ghosts (Sections 8.4 and 9).

A consumer supplies a default and a step for the stored value, which the eliminator checks at A by type application. On application, the ghost is discarded: none[a] returns the default and some[x] passes x to the step (none_apply, some_apply).

Type specification
Sum$[A, B]$ # $X \Longrightarrow$ Sm U iff
$X = F * G$ and F # $A \Longrightarrow$ Sm U and G # $B \Longrightarrow$ Sm U
inleft$[x, b]$: Sum$[A, B]$ if $x : A$ and $b : B$
inright$[a, y]$: Sum$[A, B]$ if $a : A$ and $y : B$

Type elimination
sum_eliminator$[A, B]$ @ type_app0 =
| $F * G \Rightarrow$ **let** Sm U = type_app0 @ F @$_r$ A **in**
let Sm V = type_app0 @ G @$_r$ B **in**
if equal @ U @$_r$ V **then** Sm U **else** Lf
| _ $\Rightarrow$ Lf
Sum$[A, B]$ = tagged[sum_eliminator$[A, B]$, sum_nm]

Type introduction
inleft$[x, b]$ = tagged[decorate[S_2[fst, $K_1 x$], b], sum_nm]
inright$[a, y]$ = tagged[decorate[S_2[snd, $K_1 y$], a], sum_nm]
sum_abstractor =
| decorate[S_2[fst, $K_1 A$], B] $\Rightarrow$ Sm sum_eliminator$[A, B]$
| decorate[S_2[snd, $K_1 B$], A] $\Rightarrow$ Sm sum_eliminator$[A, B]$
| _ $\Rightarrow$ Lf

Soundness
decorate[S_2[fst, $K_1 A$], B] # $(F * G) \Longrightarrow$ Sm U
decorate[S_2[snd, $K_1 B$], A] # $(F * G) \Longrightarrow$ Sm U
(F # $A \Longrightarrow$ Sm U and G # $B \Longrightarrow$ Sm U)

Figure 11. Sums

Type specification
List$[A]$ # $X \Longrightarrow$ Sm U iff $X = U * F$ and F # $(A * U) \Longrightarrow$ Sm U
nil$[a]$: List$[A]$ if $a : A$
cons$[x, t]$: List$[A]$ if $x : A$ and t : List$[A]$

Type elimination
list_eliminator$[A]$ @ type_app0 =
| $U * F \Rightarrow$ **let** Sm V = type_app0 @ F @$_r$ $(A * U)$ **in**
if equal @ V @$_r$ U **then** Sm U **else** Lf
| _ $\Rightarrow$ Lf
List$[A]$ = tagged[list_eliminator$[A]$, list_nm]

Type introduction
nil$[a]$ = tagged[decorate[fst, a], list_nm]
cell_run @ $(x * t)$ @$_r$ $(b * f) \rightsquigarrow f$ @ $(x * r)$
if t @ $(b * f) \Longrightarrow r$
cons$[x, t]$ = tagged[wait[cell_run, $x * t$], list_nm]
list_abstractor =
| decorate[fst, A] $\Rightarrow$ Sm list_eliminator$[A]$
| wait[cell_run, $A *$ List$[A]$] $\Rightarrow$ Sm list_eliminator$[A]$
| _ $\Rightarrow$ Lf

Soundness
decorate[fst, A] # $(U * F) \Longrightarrow$ Sm U
wait[cell_run, $A *$ List$[A]$] # $(U * F) \Longrightarrow$ Sm U
(F # $(A * U) \Longrightarrow$ Sm U)

Figure 12. Lists

5.4 Sums

Sums use the same device: each injection stores a value and decorates its carrier with a ghost of the absent alternative (Figure 11). So Sum$[A, B]$ is inhabited only if both A and B are (`sum_is_data`). Each injection applies the corresponding step to the stored value (`inleft_apply`, `inright_apply`).

5.5 Lists

Lists are folds, as numbers are. Each takes a base b and a step f. The empty list nil$[a]$ gives b while cons$[x, t]$ applies f to x paired with the fold of t:

$$\mathsf{nil}[a] \ @ \ (b * f) \Longrightarrow b$$
$$\mathsf{cons}[x, t] \ @ \ (b * f) \rightsquigarrow f \ @ \ (x * r) \quad \text{if } t \ @ \ (b * f) \Longrightarrow r.$$

So the eliminator checks one step: the step must take $A * U$ to U (Figure 12). An empty list has no entry from which to read its element type, so nil$[a]$ holds a ghost $a : A$ in a decoration as none$[a]$ does. Application discards the ghost, so a consumer needs no value of A.

The carrier of cons$[x, t]$ waits on cell_run with the pair $x * t$. The pair is a subtree of the carrier. So a generic query (Section 8) finds it, and the abstractor reads A from the carrier type. Like the tagger, the abstractor builds List$[A]$ at run time from A and compares it with the tail type. So List$[A]$ holds a label but is not stored in ontology. The ontology stays concrete. The other abstractors of this paper likewise store no label, so that the ontology of Section 11 is concrete (`paper_ontology_concrete`).

A list holds its tail, so a non-empty list keeps the ghost of its nil at the bottom. Two lists with the same entries can therefore differ as trees, and a generic traversal must take care with the last tail (Section 8.4). Write $[x_1, \ldots, x_k; a]$ for cons$[x_1, \ldots$ cons$[x_k,$ nil$[a]] \ldots]$ so that the ghost a at the bottom is shown.

Programs on lists are typed in the same way. A single program append serves every element type. It receives a pair $xs * ys$ and folds xs with base ys and a step cons_step that takes $x * t$ to cons$[x, t]$. It is tag-free, which is enough for it to be self-typed (`infer_tag_free`), and it takes no value of A.

Theorem 17 (`append_polymorphic`).
Suppose that ontology supports lists. Then append : append and for every element type A

$$\text{append} \ \# \ (\text{List}[A] * \text{List}[A]) \Longrightarrow \text{Sm List}[A].$$

Theorem 18 (`infer_append_application`).
Suppose that ontology supports lists. If xs : List$[A]$ and ys : List$[A]$ then append @ $(xs * ys)$ :: List$[A]$.

Thus append has the familiar polymorphism of Hindley–Milner typing [27], with the application of its type to the type of its argument in place of instantiation. Appending an empty list keeps the entries of the first list and the ghost of the second (`append_nil_right`).

Any fold needs a base of its result type. In append the second list supplies it. Elsewhere the program takes one. So

salary_eliminator @ type_app0 @$_r$ X
  = type_app0 @ X @$_r$ Nat
Salary = tagged[salary_eliminator, salary_nm]
salary[n] = tagged[swap[n], salary_nm]

employee_eliminator @ type_app0 @$_r$ X
  = type_app0 @ X @$_r$ (Nat $*$ Salary)
Emp = tagged[employee_eliminator, employee_nm]
emp[i, n] = tagged[swap[$i *$ salary[n]], employee_nm]

department_eliminator @ type_app0 @$_r$ X
  = type_app0 @ X @$_r$ (Emp $*$ List[Emp])
Dept = tagged[department_eliminator, department_nm]
dept[m, ws] = tagged[swap[$m * ws$], department_nm]

Typing
  salary[n] : Salary if n : Nat
  emp[i, n] : Emp if i : Nat and n : Nat
  dept[m, ws] : Dept if m : Emp and ws : List[Emp]

Abstractor at each name
  computed_wrap_abstractor[P] accepts only swap[T]
  where P @ Lf $\Longrightarrow$ Sm T for the payload type T

Figure 13. Salaries, employees and departments

map receives $f * (b * xs)$ where the value b of the result's element type becomes the ghost of the result.

Theorem 19 (map_polymorphic).
Suppose that ontology supports lists. Then map : map and if F # $A \Longrightarrow$ Sm B then

$$\text{map} \# (F * (B * \text{List}[A])) \Longrightarrow \text{Sm List}[B].$$

Likewise, filter_from[p, d] keeps the entries that satisfy p and ends in nil[d] with ghost d (filter_from_list).

5.6 Wrapped types and a department

An employee holds two numbers, an identifier and a salary. To raise the salaries of a department without changing the identifiers, a salary must be told apart from other numbers, and the conditionals of Section 6 tell values apart by their nametags. So a salary is a natural number with a name of its own (Figure 13). The salary salary[n] stores n in the carrier swap[n] and so hands n to any program that consumes it. The eliminator applies the argument type to Nat. So Salary # $X \Longrightarrow$ Sm U when X # Nat $\Longrightarrow$ Sm U.

Employees and departments are built in the same way, by a design pattern rather than a family. Each type gives an existing *payload type* T a name n of its own: its eliminator applies the argument type to T and its values tag the swap of a payload of type T. Call such a type a *wrapped type*. The payload type is Nat for a salary, Nat $*$ Salary for an employee and Emp $*$ List[Emp] for a department. Handing on the payload works for any payload type, abstract like Nat or a pair like Nat $*$ Salary. The abstractor at n is the *wrapper abstractor* computed_wrap_abstractor[P] for a concrete *payload program* P that computes T when applied to a dummy argument, so that P @ Lf $\Longrightarrow$ Sm T. It accepts only

Type specification
  IsNamed[n] # $U \Longrightarrow$ Sm K if U is tagged with n
  IsNamed[n] # $U \Longrightarrow$ Sm (KI) otherwise
  is_named[n] : IsNamed[n] if n is concrete

Type elimination
  IsNamed[n] = tagged[tagged_by[n], is_named_nm]

Type introduction
  is_named[n] = tagged[name_test[n], is_named_nm]
  is_named_abstractor =
    | name_test[N] $\Rightarrow$
      if is_concrete @ N **then** Sm tagged_by[N] **else** Lf
    | _ $\Rightarrow$ Lf

Soundness by values
  name_test[N] @ $v \Longrightarrow K$ if $v : U$ and U is tagged with N
  name_test[N] @ $v \Longrightarrow KI$ if $v : U$ and U is not tagged with N

Figure 14. Name testing

the carrier type swap[T]. It computes T rather than storing it, so the ontology holds no label and stays concrete (Section 11). A name with a payload type determines its wrapped type, since the abstractor accepts only swap[T]. A program finds the payload type of a name from its abstractor (get_wrap_payload_type). The ontology *supports* salaries when it maps salary_nm and type application to the wrapper abstractor of salaries, and likewise for the other wrapped types. As in the conditions of Sections 6 to 9, write wrap_type[T, n] for the wrapped type with payload type T and name n.

The employee emp[i, n] is numbered i with salary n. The department dept[m, ws] has manager m and workers ws. The department of the examples is

department_2025 =
  dept[emp[1, 58], [emp[2, 56], emp[3, 57]; emp[0, 50]]]

6 Type-Directed Conditionals

When querying or updating a structure, it is natural to search by the type. For example, to increase salaries in a department, search for sub-programs of type Salary. However, there are two difficulties. First, a value carries no type, so recognition can use only the nametag, and this must determine the type. Second, the resulting conditional will have arms of different types, namely a specialised arm for, say, salaries, and a default that is polymorphic, so no boolean can choose between them. One new abstract type answers both.

The naive solution for recognising that a value is tagged with n is name_test[n] but its application to a salary has no type (name_test_salary_untyped). The semantic difficulty is that its two answers K and KI are different types. The technical difficulty is that name_test[n] analyses its argument by triage, and type application refuses a triage at an abstract argument type such as Salary. The solution is a new abstract type IsNamed[n] with eliminator tagged_by[n] (Figure 14). The eliminator ignores the type application it receives, runs the name test on the argument type and marks the answer as

a success (Section 2.5). The semantic question then becomes one of soundness: does the answer on the type agree with the answer on each value of that type? It does, value by value. The abstractor checks that the name is concrete, so that it is self-typed and the only value of its type (Theorem 7). So a value tagged with the name has a type tagged with the same name, and the abstractor is sound by values. It is not sound by typing, which would also need the carrier type name_test$[n]$ to agree with IsNamed$[n]$ at abstract types, exactly where the naive solution failed.

IsNamed$[n]$ also resolves the second difficulty since it answers with the distinct types K and KI. The conditional is a routine definition, built from well-typed programs:

$$\text{if_named}[n, f, d] = S_2[S_2[S_2[\text{is_named}[n], K_1 f], K_1 d], I]$$
$$\text{IfNamed}[n, F, D] = S_2[S_2[S_2[\text{IsNamed}[n], K_1 F], K_1 D], I].$$

It applies is_named$[n]$ to its argument, uses the answer to select the action f or the default d and then applies the selection to the argument. The concrete clauses type S_2, K_1 and I by the same constructions, so its type IfNamed$[n, F, D]$ has the same form, and records which arm is taken.

For example, the arm salary_raise below applies a salary to the program salary_successor that wraps the successor of its argument. The arm salary_raise is tag-free, so it is self-typed, and salary_raise # Salary $\Longrightarrow$ Sm Salary. Then

$$\begin{aligned}\text{salary_raise} &= \text{swap}[\text{salary_successor}]\\ \text{salary_here} &= \text{if_named}[\text{salary_nm}, \text{salary_raise}, I]\\ \text{Salary_here} &= \text{IfNamed}[\text{salary_nm}, \text{salary_raise}, I]\end{aligned}$$

and salary_here raises salary$[5]$ to salary$[6]$ and returns every value of another type unchanged.

Theorem 20 (`salary_here_polymorphic`).
If ontology supports salaries, numbers and IsNamed then salary_here : Salary_here and Salary_here # $U \Longrightarrow$ Sm U for every inhabited type U.

In the proof, the derived rule for abstract types (Section 5.1) implies that U is Salary since U is inhabited and tagged with salary_nm and the salary abstractor accepts only the carrier type swap[Nat] (`inhabited_wrapper_tag`). Although salary_here changes some values, its type, like that of the identity, takes every inhabited argument type to itself, whereas in System F parametricity would force such a program to be the identity.

7 Arrays

The goal of this section is to type arrays by their length as well as their element type, so that appending arrays of types Array$[A, n]$ and Array$[A, m]$ gives one of type Array$[A, n + m]$. A dependent type theory would give array_append a Π-type, such as $\Pi\, n\, m.\, \text{Array}[A, n] \to \text{Array}[A, m] \to \text{Array}[A, n + m]$ for fixed A [40]. Here array_append receives only a pair of arrays. The difficulty is that the length of the result must then be computed during type application, from the types of the arrays. The idea is that the carrier of an appended array holds both arrays, so that its carrier type holds both array types, and the abstractor adds their lengths. Lengths are indices, which are trees, so type application compares them by equal. Matrix multiplication uses this to check that dimensions agree (Section 7.4).

Type specification
Array$[A, \text{Lf}]$ # $U \Longrightarrow$ Sm U
Array$[A, \text{Sm}\, n]$ # $F \Longrightarrow$ Sm W iff
  F # $A \Longrightarrow$ Sm V and Array$[A, n]$ # $V \Longrightarrow$ Sm W
empty$[w]$: Array$[A, \text{Lf}]$ if $w : A$
prepend$[a, xs]$: Array$[A, \text{Sm}\, n]$ if $a : A$ and xs : Array$[A, n]$

Type elimination
array_eliminator$[A, \text{Lf}]$ @ type_app0 $@_r$ U = Sm U
array_eliminator$[A, \text{Sm}\, n]$ @ type_app0 $@_r$ F =
  let Sm V = type_app0 @ F $@_r$ A **in**
  array_eliminator$[A, n]$ @ type_app0 $@_r$ V
Array$[A, n]$ = tagged[array_eliminator$[A, n]$, array_nm]

Type introduction
empty$[w]$ = tagged[wait$[KI, w]$, array_nm]
prepend$[a, xs]$ = tagged$[xs \circ \text{swap}[a]$, array_nm]
array_abstractor =
  | wait$[KI, A] \Rightarrow$ Sm array_eliminator$[A, \text{Lf}]$
  | Array$[A, n] \circ$ swap$[A] \Rightarrow$ Sm array_eliminator$[A, \text{Sm}\, n]$
  | Array$[A, m] \circ$ Array$[A, n] \Rightarrow$ Sm array_eliminator$[A, n + m]$
  | _ $\Rightarrow$ Lf

Soundness
wait$[KI, A]$ # $F \Longrightarrow$ Sm W if Array$[A, \text{Lf}]$ # $F \Longrightarrow$ Sm W
(Array$[A, n] \circ$ swap$[A]$) # $F \Longrightarrow$ Sm W
  if Array$[A, \text{Sm}\, n]$ # $F \Longrightarrow$ Sm W
(Array$[A, m] \circ$ Array$[A, n]$) # $F \Longrightarrow$ Sm W
  if Array$[A, n + m]$ # $F \Longrightarrow$ Sm W

Figure 15. Arrays

7.1 Array types

The indices Sm^k Lf of Section 2.5 are concrete, and so self-typed. In the length of an array type, a numeral k denotes the index Sm^k Lf. Write $n + m$ for the sum of indices n and m.

An array supplies its elements, in order, to a consumer, by one application each. An empty array returns the consumer unchanged, and holds a ghost of the element type. Prepending a applies the consumer to a and then passes the result to the remaining array. The eliminator records the same sequence of applications at the type level, with one step of type application at A for each entry, and returns what remains (Figure 15). So the type of a consumer may change at each step.

The abstractor reads A and the length from the carrier type, and adds the lengths of the two array types in the carrier type of an appended array (Section 7.2).

7.2 Array append

Array append combines polymorphism in the element type with a result length computed from its inputs. Appending arrays composes their supply of arguments:

$$\mathsf{array_append} \mathbin{@} (xs * ys) \Longrightarrow \mathsf{tagged}[ys \circ xs, \mathsf{array_nm}].$$

The carrier of its result holds both arrays, whose types supply A and the two lengths. The program is concrete, and so self-typed (`array_append_concrete`).

Being self-typed, it is applied by type application to the pair of the array types. The carrier type of the result is $\mathsf{Array}[A, m] \circ \mathsf{Array}[A, n]$ and the array abstractor (Section 7.1) computes the length $n + m$ from it (`type_app_array_append`).

Theorem 21 (`array_append_polymorphic`).
If ontology supports arrays then array_append : array_append and for every type A and all indices n and m

$$\mathsf{array_append} \# (\mathsf{Array}[A, n] * \mathsf{Array}[A, m]) \Longrightarrow \mathsf{Sm}\,\mathsf{Array}[A, n + m].$$

Theorem 22 (`infer_array_append_application`).
Suppose that ontology supports arrays. If $xs : \mathsf{Array}[A, n]$ and $ys : \mathsf{Array}[A, m]$ then $\mathsf{array_append} \mathbin{@} (xs * ys) :: \mathsf{Array}[A, n + m]$.

7.3 Reading an array

An array applies a consumer once per entry, and the type of the consumer may change at each step. So a consumer can accumulate. The array itself bounds the reading. The consumer

$$\mathsf{reading}[s, f] = \mathsf{wait}[\mathsf{array_loop}, s * f]$$

holds a state s and a step f. If $f \mathbin{@} (s * x) \Longrightarrow s'$ then its application to an entry x evaluates to $\mathsf{reading}[s', f]$ (`reading_eval`), and the final state is read back from the stored pair (`reading_result`). The program array_loop is built with Z but the array bounds it. It is tag-free and so self-typed. If $F \# (S * A) \Longrightarrow \mathsf{Sm}\,S'$ then

$$\mathsf{reading}[S, F] \# A \Longrightarrow \mathsf{Sm}\,\mathsf{reading}[S', F]$$

(`type_app_reading`). So type application takes one step per entry, as the eliminator of Figure 15 requires, with the state type changing as it goes.

One such program is array_map. Like map it receives $f * (b * xs)$ where the value b of the result's element type becomes the ghost of the result. Its state is the array built so far.

The program $\mathsf{array_rev} \mathbin{@} (b * xs)$ reverses an array, where b is the ghost of the empty array it starts from. Each entry is prepended to the state, so the type of the state counts up from $\mathsf{Array}[A, \mathsf{Lf}]$ to $\mathsf{Array}[A, n]$ one entry at a time (`type_app_array_rev`).

Theorem 23 (`array_map_polymorphic`).
If ontology supports arrays, n is an index and $F \# A \Longrightarrow \mathsf{Sm}\,B$ then

$$\mathsf{array_map} \# (F * (B * \mathsf{Array}[A, n])) \Longrightarrow \mathsf{Sm}\,\mathsf{Array}[B, n].$$

To read two arrays together, $\mathsf{array_lock}@(g*(s*(xs*ys)))$ reads xs with a state that also holds the rest of ys. It takes one entry off that rest for each entry of xs. So xs still drives the reading. Let both arrays have length n and element types A and R respectively. The typing needs a state type S_j for every tree j such that $G \# (S_j * (A * R)) \Longrightarrow \mathsf{Sm}\,S_{\mathsf{Sm}\,j}$ for every j. Then $\mathsf{array_lock}\#(G*(S_{\mathsf{Lf}}*(\mathsf{Array}[A, n]*\mathsf{Array}[R, n]))) \Longrightarrow \mathsf{Sm}\,S_n$ (`type_app_array_lock`).

7.4 Matrices

Matrix multiplication shows how type application enforces agreement between dimensions as well as determining the result dimensions. Nested arrays retain the row and column counts. For natural-number entries and positive m and n write $\mathsf{Matrix}[m, n] = \mathsf{Array}[\mathsf{Array}[\mathsf{Nat}, n], m]$. The outer index records the row count; its element type fixes the length of every row.

The program matrix_mult receives $xs * ys$. It reads each row of xs in lock step with the rows of ys. The row of the result starts as the *zero row* z with one zero for each entry of a row of ys. Each entry x of the row of xs adds x times the matching row of ys to it. A ghost cannot supply these zeros, since application discards ghosts. Positive dimensions supply them instead, since ys has a first row, whose entries the program replaces by zeros to give z. The zero row z also serves as the ghost of the result.

Theorem 24 (`infer_matrix_mult_application`).
Suppose that ontology supports arrays and numbers, and that m, n, p are positive indices. If $xs : \mathsf{Matrix}[m, n]$ and $ys : \mathsf{Matrix}[n, p]$ then

$$\mathsf{matrix_mult} \mathbin{@} (xs * ys) :: \mathsf{Matrix}[m, p].$$

To check the inner dimension, the program cannot rely on reading alone, since a row of xs that is too short leaves some rows of ys unread. So the program first appends xs to the empty array $\mathsf{empty}[w]$ whose ghost w has one zero per row of ys (where z has one per entry of a row). This ghost is never read. If $ys : \mathsf{Matrix}[k, p]$ then $w : \mathsf{Array}[\mathsf{Nat}, k]$. The array abstractor accepts the appended carrier only if the element types $\mathsf{Array}[\mathsf{Nat}, n]$ and $\mathsf{Array}[\mathsf{Nat}, k]$ are the same tree.

Theorem 25 (`matrix_mult_refused`).
Suppose that ontology supports arrays and numbers, and that n and k are positive indices with $n \neq k$. Then type application of matrix_mult to $\mathsf{Matrix}[m, n] * \mathsf{Matrix}[k, p]$ does not produce $\mathsf{Sm}\,V$ for any V.

Thus array_append and matrix_mult show dependent typing of lengths and dimensions. The programs read values, and type application computes the indices.

8 Queries

The goal of this section is to support generic queries, such as the query size that counts the nodes of any argument, and specialised queries, such as a salary bill.

Generic queries are difficult in System F, since parametricity stops a program of type $\forall X.\, X \to \mathsf{Nat}$ from reading its argument, and polymorphic recursion needs annotations in general [17]. In functional programming they are usually defined by cases on types, and so need types as parameters [19]. Here the query types are easy to specify: a query of type $\mathsf{Query}[B]$ gives a result of type B whatever its argument, and this type serves as the invariant of its recursion.

The question is which constructors to provide, and the examples decide. The query size needs constant, triage, applicative and recursive queries. The query typed_equal needs no more: after its first argument it is a query of type Query[Bool] built like size. The queries on a department need two more: a diagonal query, which looks at its argument to choose a query, and which the department queries use to skip ghosts; and a specialised query, which acts differently at salaries, built from the conditionals of Section 6. The triage query takes abstract values apart, so the query abstractor is sound by values but not by typing (Section 3.4).

8.1 Query types

The elimination rule says that applying a query of type $\mathsf{Query}[B]$ to an argument of any type gives a result of type B (Figure 16). The eliminator ignores the argument type, so it gives $\mathsf{Sm}\, B$ at every U (`type_app_Query`).

Theorem 26 (`generic_query_application`).
If $q : \mathsf{Query}[B]$ and $x : U$ then $q\ @\ x :: B$.

A query is written as an ordinary program, tagged with query_nm (Figure 16). The constant query $\mathsf{Kq}[b]$ ignores its argument. The triage query $\mathsf{triageq}[b, s, g]$ looks one level into its argument. The applicative query $\mathsf{Sq}[g, f]$ has two disjoint typing rules: either f is a query, whose result g consumes, or $f = I$ and then $\mathsf{Sq}[g, I]$ is a *diagonal* query, where g looks at the argument to choose a query and then applies it to the same argument. The recursive query *retags* itself: its body receives the recursion wrapped by tagger[query_nm] so that typing meets the recursive call at the type $\mathsf{Query}[B]$ and does not unfold it. The body is decorated with a ghost $s : \mathsf{Query}[B]$ that records its *seed* B. The specialised query applies f at values tagged with n and d elsewhere (Section 6). The query abstractor query_abstractor reads B from the carrier type of each constructor, directly or through a check by type application, and maps it to query_eliminator$[B]$. So each constructor has type $\mathsf{Query}[B]$. For the carrier type

Type specification
- $\mathsf{Query}[B]\ \#\ U \Longrightarrow \mathsf{Sm}\, B$
- $\mathsf{Kq}[b] : \mathsf{Query}[B]$ if $b : B$
- $\mathsf{triageq}[b, s, g] : \mathsf{Query}[B]$ if $b : B$ and $s : \mathsf{Query}[B]$ and $g : \mathsf{Query}[\mathsf{Query}[B]]$
- $\mathsf{Sq}[g, f] : \mathsf{Query}[B]$ if $g : \mathsf{Query}[F]$ and $f : \mathsf{Query}[V]$ and $F\ \#\ V \Longrightarrow \mathsf{Sm}\, B$
- $\mathsf{Sq}[g, I] : \mathsf{Query}[B]$ if $g : \mathsf{Query}[\mathsf{Query}[B]]$
- fix_query$[$decorate$[f, s]] : \mathsf{Query}[B]$ if $f : F$ and $s : \mathsf{Query}[B]$ and $F\ \#\ \mathsf{Query}[B] \Longrightarrow \mathsf{Sm}\ \mathsf{Query}[B]$
- specialised_query$[n, f, d] : \mathsf{Query}[B]$ if $d : \mathsf{Query}[B]$ and $f : F$ and $F\ \#\ X \Longrightarrow \mathsf{Sm}\, B$ and $X =$ wrap_type$[T, n]$ and n is concrete with payload type T and IsNamed is supported

Type elimination
- query_eliminator$[B]$ @ type_app0 $@_r\ U = \mathsf{Sm}\, B$
- $\mathsf{Query}[B] =$ tagged[query_eliminator$[B]$, query_nm]

Type introduction
- $\mathsf{Kq}[b] =$ tagged$[K_1 b$, query_nm$]$
- $\mathsf{triageq}[b, s, g] =$ tagged[triage$[b, s, g]$, query_nm$]$
- $\mathsf{Sq}[g, f] =$ tagged$[S_2[g, f]$, query_nm$]$
- fix_query$[f] =$ tagged$[Z(f \circ$ tagger[query_nm$])$, query_nm$]$
- specialised_query$[n, f, d] =$ tagged[if_named$[n, f, d]$, query_nm$]$
- query_abstractor[ontology] @ type_app0 =
  - $\mid K_1 B \Rightarrow \mathsf{Sm}$ query_eliminator$[B]$
  - $\mid$ triage$[B, \mathsf{Query}[B], \mathsf{Query}[\mathsf{Query}[B]]] \Rightarrow$ the same
  - $\mid S_2[\mathsf{Query}[F], \mathsf{Query}[V]] \Rightarrow$ the same, if $F\ \#\ V \Longrightarrow \mathsf{Sm}\, B$
  - $\mid S_2[\mathsf{Query}[\mathsf{Query}[B]], I] \Rightarrow$ the same
  - $\mid Z(\mathsf{decorate}[F, \mathsf{Query}[B]] \circ \mathsf{tagger}[\mathsf{query_nm}]) \Rightarrow$ the same, if $F\ \#\ \mathsf{Query}[B] \Longrightarrow \mathsf{Sm}\ \mathsf{Query}[B]$
  - $\mid$ IfNamed$[n, F, \mathsf{Query}[B]] \Rightarrow$ the same, if n is concrete with payload type T and $F\ \#\ \mathsf{wrap_type}[T, n] \Longrightarrow \mathsf{Sm}\, B$
  - $\mid _ \Rightarrow \mathsf{Lf}$

Soundness
- $C\ \#\ U \Longrightarrow \mathsf{Sm}\, B$ for the carrier types of K_1, S_2 and Z above
- IfNamed$[n, F, \mathsf{Query}[B]]\ \#\ U \Longrightarrow \mathsf{Sm}\, B$ (U inhabited)
- the triage case is sound by values only

Figure 16. Query types. Typing of a recursive query meets its recursive call at $\mathsf{Query}[B]$ and does not unfold it.

of the specialised query the abstractor also checks that n is concrete by is_concrete and finds the payload type T of n from the ontology by get_wrap_payload_type (Section 5.6). So the query abstractor takes the ontology as a parameter, as query_abstractor[ontology].

The carrier types of the constant, applicative and recursive queries give B at every argument type, and that of the specialised query gives B at inhabited types. This is because the only inhabited type tagged with n is wrap_type$[T, n]$ when n has the payload type T (`inhabited_not_impostor`), and at that type the abstractor has checked f.

The exception is the triage query, which looks one level down. Its carrier has a leaf case of type B; a stem case, which accepts one child, of type $\mathsf{Query}[B]$; and a fork

case, which accepts two children in succession, of type Query[Query[B]]. Its carrier type gives B at argument types that are not abstract (`installed_traversal`), but not at abstract ones, which the triage clause of type application refuses to inspect (Figure 5). So the query abstractor is not sound by typing (`queries_not_sound_by_typing`). It is still sound by values (Section 3.4), because the triage takes any typed value apart and each case returns a value of type B. Here the premise EPT(n) does the work. On a fork with children x and y the carrier applies a query g : Query[Query[B]] to x and then to y. The premise EPT(n) gives the values of these applications their types Query[B] and B because their evaluations lie within that of the carrier (`query_triage_reduct_typing`, `query_one_level_value_typing`).

8.2 Size and equality

Size counts the nodes of a value tree by the usual algorithm [21]: a leaf has size one; a stem the successor of its child's size; and a fork the successor of the sum of its children's sizes. If the recursive parameter has type Query[Nat] then the three cases have types Nat, Query[Nat] and Query[Query[Nat]]. The stem and fork cases combine the recursive results by applicative queries. A triage query triageq assembles the three cases into a body size_functional_body whose type F satisfies F # Query[Nat] $\Longrightarrow$ Sm Query[Nat]. Then size = fix_query[decorate[size_functional_body, Kq[1]]] with the seed Kq[1].

Theorem 27 (`infer_size`).
If ontology supports queries and numbers, then size : Query[Nat].

So size @ t :: Nat whatever the type of t (Theorem 26): size measures data, functions, queries and the inference program alike, and even the snapshots of a running computation (Section 12.4).

Equality compares the outer shapes of two trees, then their children recursively [21]. Unlike equal of Section 2.3, it returns tt or ff. So it is called typed_equal. After receiving one tree it is a query in the other, so its type is Query[Query[Bool]]. It is built from the same constructors as size.

Theorem 28 (`typed_equal_eval`).
The expression typed_equal @ x @$_r$ y evaluates to tt if the trees x and y are equal, and to ff otherwise.

The arguments need not have the same type, so it compares functions as programs, which typed languages usually forbid.

Theorem 29 (`typed_equal_polymorphic`).
If ontology supports queries and booleans then typed_equal : Query[Query[Bool]] and Query[Query[Bool]] # U # V $\Longrightarrow$ Sm Bool for all U and V and typed_equal @ typed_equal @$_r$ typed_equal $\Longrightarrow$ tt.

cdither = $Z(\lambda r.\,\mathsf{cneg} \circ r)$
concrete: typing does not finish

zdither = $Z(\lambda r.\,\mathsf{neg} \circ r)$
with booleans: type application does not finish

dither = fix_query[decorate[d, Kq[tt]]]
invariant Query[Bool]: dither @ tt :: Bool, with no value

Figure 17. Three recursions that negate their own results, and where their typing stops

8.3 Dither

The recursions cdither and zdither of Sections 4 and 5.2 negate their own results, and their typing does not finish (Theorems 13 and 16). As a query, the same recursion is typed. Let dither = fix_query[decorate[d, Kq[tt]]] where the body d maps the recursive call r to the query Sq[Kq[neg], r] that negates the result of r. The type of Kq[neg] is Query[Neg] and Neg # Bool $\Longrightarrow$ Sm Bool. So the body takes Query[Bool] to Query[Bool] and dither : Query[Bool] (`infer_dither`).

Theorem 30 (`dither_typed_no_value`).
If ontology supports queries and booleans then dither @ tt :: Bool and yet dither @ tt has no value.

Each recursive call must finish before its result can be negated, and every call makes another. The invariant promises the type of the result, not that there is one (Figure 17).

This is the liar "This sentence is false". As in Kripke's account of the liar [25], dither gives a gap rather than a contradiction. Soundness still holds, since it constrains only the values that evaluation produces.

8.4 Querying a department

Generic queries see everything, including ghosts, so a headcount built like size would also count the ghost employee at the end of a list. The diagonal query cons_case[b, q, h] tests the tag of a list by nested triage queries. At nil[a] it gives b and never reads the ghost; at cons[x, t] it applies q to $x * t$; and at any value not tagged with list_nm it applies h. If ontology supports booleans and queries and $b : B$ and q : Query[B] and h : Query[B] then cons_case[b, q, h] : Query[B] (`cons_case_typed`). Decorations hold ghosts too: the value w stored by a decoration decorate[f, w] is a ghost. The diagonal query ghost_case[b, q, h] avoids such ghosts. At a decoration decorate[f, w] it applies q to the program f alone, since f may store data, and at any other value it applies h. The value b never reaches the result. It is there only for typing. If ontology supports booleans and queries and $b : B$ and

q : Query$[B]$ and h : Query$[B]$ then ghost_case$[b, q, h]$: Query$[B]$ (ghost_case_typed).

The headcount count_employees has type Query[Nat] for the sound ontology paper_ontology of Section 11, which supports the lists, wrapped types, queries and conditionals that count_employees mixes (count_employees_typed). It adapts size: cons_case and ghost_case skip the ghosts, a specialised query makes each employee contribute one, and otherwise the counts are summed.

Theorem 31 (count_employees_department_example).
count_employees @ department_2025 $\Longrightarrow$ 3.

The salary bill is built in the same way, except that each salary contributes its amount.

Theorem 32 (salary_bill_department_2025).
salary_bill @ department_2025 $\Longrightarrow$ 171.

A bill built without cons_case and ghost_case reads the ghost emp$[0, 50]$ at the end of the workers, and gives 221 (naive_salary_bill_department_2025). On a department of one manager with the empty list nil[emp$[0, 50]$] of workers it likewise includes the salary of 50 in its total of 108 (naive_salary_bill_empty), whereas salary_bill gives 0 on an empty list (salary_bill_nil).

Queries can also select. The query paid_over$[t, m, a]$ collects the employees whose salary is at least t. It stores its threshold as salary$[t]$ within the program tree, where a salary update can reach it (Section 9.3). Its other two parameters are ghosts, which stand in for the types of employees and salaries. The *ghost employee* a ends its result nil$[a]$ and decorates its test of a salary. The *ghost salary* salary$[m]$ lies in the ghost $0 * \text{salary}[m]$ of the decoration by which it reads a salary. Write paid_over$[t]$ for paid_over$[t, 0, \text{emp}[0, 0]]$.

Theorem 33 (paid_over_typed).
For paper_ontology if a : Emp then paid_over$[t, m, a]$: Query[List[Emp]] for all numerals t and m.

Theorem 34 (paid_over_57_original).

paid_over[57] @ department_2025
$\Longrightarrow$ [emp$[1, 58]$, emp$[3, 57]$; emp$[0, 0]$].

9 Updates

The goal of this section is to support generic and specialised updates, such as one that gives every employee of a department a rise. An update changes selected values in data, programs or expressions while preserving the type of the whole argument.

Generic updates are difficult in System F, since parametricity forces a program of type $\forall X. X \to X$ to be the identity [31, 39]. In functional programming they are usually defined by cases on types, and so need types as parameters [19]. Here the update type is easy to specify: Update # $U \Longrightarrow \text{Sm}\, U$. So an update applies like the identity although it need not be one, and this type serves as the invariant of its recursion.

Again the question is which constructors to provide, and the examples decide. Raising every salary needs recursive, specialised and children updates. Doing so without touching ghosts needs a query update, which lets a diagonal query of Section 8 choose the update. Minimisation needs the identity update, for the arm that stops. The children update rebuilds abstract values from their parts, so the update abstractor is sound by values but not by typing (Section 3.4).

Type specification
- Update # $U \Longrightarrow \text{Sm}\, U$
- identity_update : Update
- children$[r]$: Update if r : Update
- specialised_update$[n, f, d]$: Update if d : Update
  - and $f : F$ and $F \# X \Longrightarrow \text{Sm}\, X$
  - and X = wrap_type$[T, n]$
  - and n is concrete with payload type T
  - and IsNamed is supported
- query_update$[q]$: Update if q : Query[Update]
- fix_update$[b]$: Update if $b : F$
  - and F # Update $\Longrightarrow$ Sm Update

Type elimination
- update_eliminator @ type_app0 @$_r$ U = Sm U
- Update = tagged[update_eliminator, update_nm]

Type introduction
- identity_update = tagged$[I$, update_nm$]$
- children$[r]$ = tagged[update_one_level$[r]$, update_nm$]$
- specialised_update$[n, f, d]$ =
  - tagged[if_named$[n, f, d]$, update_nm$]$
- query_update$[q]$ = tagged$[S_2[q, I]$, update_nm$]$
- fix_update$[b]$ = tagged$[Z(b \circ \text{tagger}[\text{update_nm}])$, update_nm$]$
- update_abstractor[ontology] @ type_app0 =
  - $\mid I \Rightarrow$ Sm update_eliminator
  - $\mid$ update_one_level[Update] $\Rightarrow$ the same
  - $\mid$ IfNamed$[n, F,$ Update$] \Rightarrow$ the same, if n is concrete with payload type T and $F \# X \Longrightarrow \text{Sm}\, X$
  - $\mid S_2[$Query[Update]$, I] \Rightarrow$ the same
  - $\mid Z(F \circ \text{tagger}[\text{update_nm}]) \Rightarrow$ the same, if F # Update $\Longrightarrow$ Sm Update
  - $\mid _ \Rightarrow$ Lf

Soundness
- $C \# U \Longrightarrow \text{Sm}\, U$ for the carrier types of I, S_2 and Z
- IfNamed$[n, F,$ Update$] \# U \Longrightarrow \text{Sm}\, U$ (U inhabited)
- the children case is sound by values only

Figure 18. Update types. Typing of a recursive update meets its recursive call at Update and does not unfold it.

9.1 Update types

In Figure 18 the eliminator maps each argument type U to Sm U (type_app_Update). Hence u @ $x :: U$ whenever u : Update and $x : U$.

An update is written as an ordinary program, tagged with update_nm (Figure 18). The identity update changes nothing. The children update rebuilds its argument from its updated children (below). The specialised update applies f at values tagged with n and d elsewhere (Section 6). A query update applies the update chosen by a query to the original argument, as the diagonal query does (Section 8.1). The recursive update retags itself, as the recursive query does. Since Update has no parameter, a recursive update needs no seed. The update abstractor maps the carrier type of each constructor to update_eliminator. So each constructor has type Update. Like the query abstractor, the update abstractor update_abstractor[ontology] takes the ontology as a parameter and finds payload types in it.

The carrier types of the identity, query and recursive updates behave like Update and so does that of the specialised update at inhabited argument types, for the reason given for queries (Section 8.1).

The exception is the children update. To descend one level, it rebuilds the parent from the results of applying an update r to each child:

$$\mathsf{update_one_level}[r] = \mathsf{triage}[\mathsf{Lf}, \mathsf{Lf} \circ r, \mathsf{update_fork}[r]]$$

where $\mathsf{update_fork}[r] \mathbin{@} x \mathbin{@}_r y$ evaluates to $\mathsf{Fk}\, x'\, y'$ when $r \mathbin{@} x \Longrightarrow x'$ and $r \mathbin{@} y \Longrightarrow y'$. If r : Update then the carrier's type is obtained by replacing r by Update. As for queries, type application refuses to apply it to an abstract argument type, so the update abstractor is not sound by typing (updates_not_sound_by_typing). It is still sound by values, because each child keeps its type, and the type of a fork is computed from the types of its children (children_reduct_typing).

Theorem 35 (abstract_update_preserves_typing).
If ontology is sound by values and u : Update and $x : U$ and $u \mathbin{@} x \Longrightarrow y$ then $y : U$.

Here U may be a data type, a query type or, later, a function type. If U is concrete then $y = x$ because a concrete type has only one value (Theorem 7). So updates change values only where abstract types are involved.

The update $\mathsf{update}[n, f]$ acts wherever a value tagged with n occurs. It is the recursive update $\mathsf{fix_update}[b]$ whose body b applies f at a value tagged with n and elsewhere descends with $\mathsf{children}[r]$ where r is the recursive call:

$$b \mathbin{@} r \Longrightarrow \mathsf{specialised_update}[n, f, \mathsf{children}[r]].$$

If ontology supports updates and IsNamed and n is concrete with payload type T and $f : F$ where $F \# X \Longrightarrow \mathsf{Sm}\, X$ and $X = \mathsf{wrap_type}[T, n]$ then $\mathsf{update}[n, f]$: Update (infer_update). For example:

Theorem 36 (infer_update_salary).
If ontology supports salaries, updates and IsNamed and $f : F$ where $F \# \mathsf{Salary} \Longrightarrow \mathsf{Sm}\, \mathsf{Salary}$ then $\mathsf{update}[\mathsf{salary_nm}, f]$: Update.

9.2 Raising salaries

Updates also see ghosts. The naive salary update is $\mathsf{update}[n, f]$ at n = salary_nm and f = salary_raise of Section 6. It increments every salary in department_2025 (department_example_updated), including that of the ghost at the end of the workers. On $\mathsf{nil}[a]$ it likewise raises the salary of the ghost employee a (salary_update_nil), and it does the same to the ghosts of decorations (salary_update_decorate).

Two query updates avoid ghosts. First, $\mathsf{cons_update}[k, h]$ leaves $\mathsf{nil}[a]$ alone, rebuilds $\mathsf{cons}[x, t]$ around k applied to $x * t$ and applies h to any value not tagged with list_nm. Second, $\mathsf{ghost_update}[k, h]$ uses ghost_case (Section 8.4): at a decoration it applies k to the program and keeps the ghost, which the decoration needs for its type; and at any other value it applies h. It rebuilds the decoration by a variant of the children update that uses separate updates for a stem and for the two branches of a fork. If all three have type Update then the variant has the same carrier type as the children update, so it needs no constructor of its own (inf_children2).

The salary update salary_update is the naive one with these two updates around its recursive step: its body b' satisfies

$$b' \mathbin{@} r \Longrightarrow \mathsf{cons_update}[r, \mathsf{ghost_update}[r, \mathsf{specialised_update}[n, f, \mathsf{children}[r]]]]$$

at n = salary_nm and f = salary_raise.

Theorem 37 (salary_update_department_2025).

$$\mathsf{salary_update} \mathbin{@} \mathsf{department_2025} \Longrightarrow \mathsf{dept}[\mathsf{emp}[1, 59], [\mathsf{emp}[2, 57], \mathsf{emp}[3, 58]; \mathsf{emp}[0, 50]]].$$

Call the result department_2026. Every employee gets a rise, including the manager, and an empty list would keep its ghost unchanged (su_nil).

9.3 Patching a query program

Ordinarily, editing a program is a matter for quotation and staging [35]: the program is first represented as data, then edited. Here a program is a tree, whose role determines whether it is run or read, so an update edits a program as it stands. The salary update reaches the programs that query the department as readily as the department itself. The query $\mathsf{paid_over}[57]$ of Section 8.4 stores its threshold as a salary inside the program, so the salary update reaches it without executing the query. The salary update raises every salary it reaches, so the programmer chooses which programs to expose to it.

Theorem 38 (salary_update_polymorphic).
For paper_ontology the program salary_update has type Update and Update # Dept $\Longrightarrow$ Sm Dept and $\mathsf{salary_update} \mathbin{@} \mathsf{paid_over}[57] \Longrightarrow \mathsf{paid_over}[58]$.

By Theorem 35 the result keeps its type Query[List[Emp]] (paid_over_58_typed). The update leaves ghosts alone, so the ghost employee in the query stays at emp[0, 0].

9.4 Minimisation

Minimisation was refused in Section 5.2 because the arm of its conditional that stops and the arm that continues have different types. Written as an update, both arms have type Update. A specialised update acts at the name of a wrapped type, so the search runs on the wrapped type Seek = wrap_type[Nat, seek_nm]. The ontology supports seeks when ontology @ seek_nm $\Longrightarrow$ K_1computed_wrap_abstractor[salary_payload_program]. This is the wrapper abstractor of Salary (Section 5.6), which serves Seek too since both payloads are Nat. The value seek_at[n] marks a search in progress at n. Given a predicate p the update minimisation @ p is fix_update[b] where

$$b \mathbin{@} r \Longrightarrow \text{specialised_update}[\text{seek_nm}, \text{step}[p, r], \text{identity_update}]$$

and the test p @ n makes the same choice twice when x = seek_at[n]:

$$\text{step}[p, r] \mathbin{@} x \rightsquigarrow (p \mathbin{@} n) \mathbin{@} (\text{identity_update} * r) \mathbin{@} ((p \mathbin{@} n) \mathbin{@} (x * \text{seek_at}[\text{succ}[n]])).$$

If p holds at n then the identity is applied to x and otherwise the recursive call r is applied to seek_at[succ[n]]. Each choice is between two values of one type, Update or Seek. So the conditionals are typed, and typing meets r at Update and does not unfold it. The program minimisation builds its tagged parts only when applied, so it has no tagged subtree and is self-typed. Its polymorphism in the predicate is revealed by type application.

Theorem 39 (minimisation_polymorphic).
If ontology supports seeks, updates, IsNamed, booleans and numbers and P # Nat $\Longrightarrow$ Sm Bool then minimisation : minimisation and minimisation # P $\Longrightarrow$ Sm Update.

For example, let is_odd be the fold that tests whether a number is odd.

Theorem 40 (minimise_is_odd).
If ontology supports seeks, updates, IsNamed, booleans and numbers then minimisation @ is_odd @$_r$ seek_at[0] $\Longrightarrow$ seek_at[1] and minimisation@is_odd@$_r$seek_at[0] :: Seek.

The search operator nat_search of Figure 1 hides the wrapper: nat_search @ p @$_r$ n wraps n as seek_at[n] and applies minimisation @ p before unwrapping the result by applying it to I. It has no tagged subtree, and so is self-typed.

Theorem 41 (nat_search_polymorphic).
If ontology supports seeks, updates, IsNamed, booleans and numbers and P # Nat $\Longrightarrow$ Sm Bool then nat_search : nat_search and nat_search # P # Nat $\Longrightarrow$ Sm Nat.

For example, nat_search @ is_odd @$_r$ 0 $\Longrightarrow$ 1 with type Nat (nat_search_is_odd). The update promises the type of its result and not that there is one, as dither does. The recursions whose typing did not finish (Sections 4 and 5.2) or was refused (Section 5.2) are now typed by invariants: self-negation by the query dither and minimisation by an update.

Minimisation completes the μ-recursive functions of Theorem 11, now on the numbers of Section 5.2. Composition is concrete, and primitive recursion is a fold of a number over a triple of counter, accumulator and arguments, whose step type the eliminator of Nat checks once. The arguments form a single tuple of type $\text{Nat}^k = \text{Nat} * (\cdots * \text{Nat})$.

Theorem 42 (typed_mu_complete).
Let f be a μ-recursive function of positive arity k. For paper_ontology there is a program P of type R such that R # Nat^k $\Longrightarrow$ Sm Nat whether or not f is total, and if f maps arguments $\vec{x}$ to y then P applied to the tuple of numerals of $\vec{x}$ evaluates to the numeral of y.

Here R is P itself, which has no tagged subtree. The type application finishes since it stops at the invariants and never runs f. By Theorem 5 every value of P on numerals is a number (typed_mu_values_are_numbers). So the partial recursive functions are typed by invariant, where the concrete programs of Theorem 11 are self-typed, each program its own type.

10 Functions

The main examples of the paper have been developed with types F such that F # U $\Longrightarrow$ Sm V in place of function types $U \to V$. Their recursions stop at query or update types, as for dither and minimisation. The goal of this section is to recover the traditional rule $(U \to V)$ # U $\Longrightarrow$ Sm V by adding function types as one more family of abstract types. Future work may use them to represent logical propositions (Appendix A). Then minimisation has the textbook type (Nat → Bool) → (Nat → Nat) of the introduction, and a query on a department has the ordinary interface Dept → List[Emp]. The difficulty is that a function type records only a domain and a codomain, which many programs share, whereas each program has its own type F. The idea is to forget F and keep U and V. A lambda program supplies the domain by a ghost (Section 10.1), and a recursive function retags its recursive call, as a recursive query does (Section 10.2).

10.1 Function types

Traditional type theory begins with function types, but here they come last. If a program has type F such that

$$F \# U \Longrightarrow \text{Sm}\, V$$

then $U \to V$ keeps the domain U and the codomain V and forgets F. This can lose polymorphism too: the type $K_1 W$ of

Type specification

$(U \to V)\ \#\ X \Longrightarrow \mathsf{Sm}\ W$ iff $X = U$ and $W = V$

$\mathsf{lam}[f, u] : U \to V$

if $f : F$ and $u : U$ and $F\ \#\ U \Longrightarrow \mathsf{Sm}\ V$

$\mathsf{fix_function}[f] : U \to V$

if $f : (U \to V) \to (U \to V)$

Type elimination

$\mathsf{funty_elim}\ @\ (U * V)\ @_r\ \mathsf{type_app0}\ @_r\ X =$

if $\mathsf{equal}\ @\ U\ @_r\ X$ **then** $\mathsf{Sm}\ V$ **else** Lf

$\mathsf{funty_eliminator}[U, V] = \mathsf{wait}[\mathsf{funty_elim}, U * V]$

$U \to V = \mathsf{tagged}[\mathsf{funty_eliminator}[U, V], \mathsf{fun_nm}]$

Type introduction

$\mathsf{lam}[f, u] = \mathsf{tagged}[\mathsf{decorate}[f, u], \mathsf{fun_nm}]$

$\mathsf{fix_function}[f] =$

$\mathsf{tagged}[Z(f \circ \mathsf{tagger}[\mathsf{fun_nm}]), \mathsf{fun_nm}]$

$\mathsf{fix}[U, V] = Z(((U \to V) \to (U \to V)) \circ \mathsf{tagger}[\mathsf{fun_nm}])$

$\mathsf{function_abstractor}\ @\ \mathsf{type_app0}\ @_r\ C =$ **match** C **with**

$|\ \mathsf{decorate}[F, U] \Rightarrow$

let $\mathsf{Sm}\ V = \mathsf{type_app0}\ @\ F\ @_r\ U$ **in**

$\mathsf{Sm}\ \mathsf{funty_eliminator}[U, V]$

$|\ \mathsf{fix}[U, V] \Rightarrow \mathsf{Sm}\ \mathsf{funty_eliminator}[U, V]$

$|\ _ \Rightarrow \mathsf{Lf}$

end

Soundness

$\mathsf{decorate}[F, U]\ \#\ U \Longrightarrow \mathsf{Sm}\ V \quad (F\ \#\ U \Longrightarrow \mathsf{Sm}\ V)$

$\mathsf{fix}[U, V]\ \#\ U \Longrightarrow \mathsf{Sm}\ V$

Figure 19. Function types. Typing of a recursive function meets its recursive call at $U \to V$ and does not unfold it.

$K_1 w$ accepts every argument type, whereas $U \to W$ accepts only U.

Since many functions share this type, function types must be abstract. Query and update types are shared too. What function types add is a recorded domain. A list of different numeric functions can then have a type such as List[Nat → Nat] and so can a table of the passes of a tool (Section 12), although each program has its own type. The values of type $U \to V$ are constructed from a program of some such F (Figure 19), so function type introduction uses the type application that ontology passes to the abstractor function_abstractor. The program ontology supports function types when ontology @ fun_nm ⟹ function_abstractor. There is no K_1 since function_abstractor uses type application. Lambda programs recur among the examples, as in emptied and minimisation_fn below, so this support is the default of Section 3, and the Theorems that use it do not repeat it.

The body of a recursive function has a function type too. So some values of function type must be built without recursion. These are the *lambda programs* $\mathsf{lam}[f, u]$ with *action* f and ghost u (Figure 19). The ghost determines the domain, as for $\mathsf{none}[a]$. The codomain is then computed by type application. The ghost is a value rather than the type U itself, because the type of a decoration comes from the type of what it stores, and types need not have types (Bool has none when ontology supports booleans, by `Bool_untypable`). So a lambda program has a function type only when its domain is inhabited. On application the ghost is discarded: $\mathsf{lam}[f, u]\ @\ x \rightsquigarrow f\ @\ x$.

Theorem 43 (`infer_lam`).
If $f : F$ and $u : U$ and $F\ \#\ U \Longrightarrow \mathsf{Sm}\ V$ then $\mathsf{lam}[f, u] : U \to V$.

The first soundness condition of Figure 19 shows that function_abstractor is sound by typing at lambda programs.

A function acquires a function type by decoration. The negation neg of Section 5.1 has type Neg. This is not a function type. If ontology supports booleans then decorating it gives typed_neg = lam[neg, tt] : Bool → Bool (`typed_neg_infer`). Similarly, the lambda program increment_salary with ghost salary[0] raises salaries, with increment_salary@ salary[n] ⟹ salary[succ[n]]. If ontology supports salaries and numbers then increment_salary : Salary → Salary.

A program that returns a list of the same type can find the base of its fold in its argument. The program emptied gives an empty list of the type of its argument, with emptied @ $\mathsf{nil}[a] \Longrightarrow \mathsf{nil}[a]$ and emptied @ $\mathsf{cons}[x, t] \Longrightarrow \mathsf{nil}[x]$. It folds a list xs to a lambda program with ghost xs and then applies that program to xs. The fold gives the identity at $\mathsf{nil}[a]$ and the constant function of $\mathsf{nil}[x]$ at $\mathsf{cons}[x, t]$. The ghost xs gives each the type of the list as its domain. The programs list_tail and filter[p] use emptied for their base, and so need no value of the element type. The first removes the head of a list, and the second filters a list by p.

Updates reach lambda programs too. The naive salary update of Section 9.2 raises the salaries in both the action and the ghost of a lambda program (`salary_update_lam`). The query paid_over[57] of Section 8.4 can be given the ordinary function type Dept → List[Emp] by the lambda program paid_at_least[57] = lam[f, w]. Its action f stores the threshold salary[57] and its ghost is a department w.

Theorem 44 (`salary_update_paid_at_least_57`).
$\mathsf{salary_update}\ @\ \mathsf{paid_at_least}[57] \Longrightarrow \mathsf{paid_at_least}[58]$.

The result has the same type (`paid_at_least_58_typed`).

10.2 Recursive functions

A body f of type $(U \to V) \to (U \to V)$ receives the recursive call at the invariant $U \to V$ and returns a function of type $U \to V$. The recursive call must be a tagged value so that typing meets it at the type $U \to V$ and does not unfold it, as for fix_query. So fix_function[f] passes the tagged recursive function back to its body, by the tagger tagger[fun_nm] (Figures 4 and 19). Write $r = Z(f \circ \mathsf{tagger}[\mathsf{fun_nm}])$ and $h = \mathsf{fix_function}[f]$. Composition retags r as h:

$$h\ @\ x \rightsquigarrow r\ @\ x \rightsquigarrow f\ @\ h\ @_r\ x.$$

The program r has type $\mathsf{fix}[U, V]$ since composition, Z and the tagger are concrete, so the type of r is r itself with f replaced by the type of f. The second case of function_abstractor recognises this shape. The body's type already supplies the domain and codomain, so this case needs no ghost and no call to type application.

Theorem 45 (`infer_fix_function`).
If $f : (U \to V) \to (U \to V)$ then $\mathsf{fix_function}[f] : U \to V$.

Type application at U follows the construction: retagging produces $U \to V$; the body's type maps this to $U \to V$; and application produces V. This is the second soundness condition of Figure 19. With the first, it shows that function type introduction is sound by typing, whatever the other families.

Theorem 46 (`function_abstractor_sound`).
If function_abstractor @ type_app $@_r\ C \Longrightarrow \mathsf{Sm}\, E$ and E @ type_app $@_r\ U \Longrightarrow \mathsf{Sm}\, V$ then $C \,\#\, U \Longrightarrow \mathsf{Sm}\, V$.

The recursions dither and minimisation can be written this way too. The function dither_fn = fix_function$[d]$ has a body d that composes typed_neg after its recursive argument. If ontology supports booleans then dither_fn @ tt :: Bool and has no value, as for the query dither (`dither_fn_typed_no_value`). The function minimisation_fn is a lambda program with ghost $\mathsf{lam}[K_1\mathsf{tt}, \mathsf{zero}]$. Given a predicate, it gives a recursive function that searches upwards from its argument.

Theorem 47 (`infer_minimisation_fn`).
If ontology supports booleans and numbers then minimisation_fn : (Nat → Bool) → (Nat → Nat).

The predicate is_odd needs a function type, so it is given a ghost: typed_is_odd = $\mathsf{lam}[\mathsf{is_odd}, 0]$: Nat → Bool. Then minimisation_fn @ typed_is_odd $@_r\ 0 \Longrightarrow 1$ and minimisation_fn @ typed_is_odd $@_r\ 0$ :: Nat (`infer_minimise_is_odd_fn`). Compared with nat_search (Section 9.4), the function type records the domain Nat but the predicate needs a ghost.

11 A Sound Ontology

Figure 20 gathers all of the abstractors built in this paper into a single ontology. With it, evaluation preserves typing and infer is self-typed. The query and update abstractors receive paper_ontology itself, from which they read payload types. The fixfunction Z ties this self-reference. Here paper_ontology is Z applied to a program that receives the ontology and adds the query and update cases to the others (`paper_ontology_from_table`, `generic_ontology_body_eval`).

The program paper_ontology is sound by values (`paper_ontology_sound`).

Theorem 48 (`paper_evaluation_preserves_typing`).

```
paper_ontology @ N @_r type_app0 =
| query_nm ⇒
    query_abstractor[paper_ontology] @ type_app0
| update_nm ⇒
    update_abstractor[paper_ontology] @ type_app0
| is_named_nm ⇒ is_named_abstractor
| employee_nm ⇒
    computed_wrap_abstractor[employee_payload_program]
| department_nm ⇒
    computed_wrap_abstractor[department_payload_program]
| salary_nm ⇒
    computed_wrap_abstractor[salary_payload_program]
| seek_nm ⇒
    computed_wrap_abstractor[salary_payload_program]
| array_nm ⇒ array_abstractor
| list_nm ⇒ list_abstractor
| sum_nm ⇒ sum_abstractor
| opt_nm ⇒ option_abstractor
| nat_nm ⇒ nat_abstractor
| bool_nm ⇒ bool_abstractor
| fun_nm ⇒ function_abstractor @ type_app0
| _ ⇒ K_1 Lf
```

Figure 20. The ontology of this paper. Seek shares the payload program of Salary because both payloads are Nat.

Evaluation preserves typing for paper_ontology: if $e :: T$ and $e \Longrightarrow v$ then $v : T$.

Since paper_ontology supports every family of Sections 5 to 10, their typings hold for it, including Theorem 42.

The program paper_ontology is also concrete (`paper_ontology_concrete`), so Theorem 12 applies.

Theorem 49 (`paper_infer_self_typed`).
If infer is built using paper_ontology then infer @ infer $\Longrightarrow$ Sm infer (Figure 2).

So the typing program lies within the typed, concrete fragment that it describes. Its type sets the next task. Since infer inspects its argument by triage, type application of its type, infer itself, refuses abstract argument types such as Bool (`type_app_infer_Bool`), even though infer evaluates on values of those types. A query type for infer would accept every argument type and is the candidate of Appendix A.

12 Towards Typed Tooling

The goal of this section is to show that tools can be typed programs in the calculus, acting on programs, on the expressions that run them, and on the snapshots of their evaluation (Figure 21). Ordinarily such tools sit outside the language and its type system, since code must first be represented as data. Here programs, expressions and snapshots are all trees, so a tool is applied to them as they stand, and its type says what it preserves.

The question is what each tool needs of typing. An update preserves value types, so patching an expression needs only soundness by values (Section 12.1). An optimiser preserves

size : Query[Nat]
  e :: T implies size @ e :: Nat
  size @ snapshot[92k] (dither @ tt) $\Longrightarrow$ 339 + 86k

salary_update @ (paid_over[57] @ department_2025)
  :: Query[List[Emp]] @ Dept

optimise : optimise
  optimise @ (Kq[0] @ tt) $\Longrightarrow$ $K_1 0$ @ tt

Figure 21. Tooling as programming with expressions. Here snapshot[j] e is the expression that e reaches after j steps, and Query[List[Emp]] @ Dept is the type of the tree paid_over[57] @ department_2025 taken as a value.

only expression types, and so needs soundness by typing at the carriers it exposes. Safe heads ensure it (Section 12.2). A second transformation, specialising equality, trades space for time, and size measures the trade (Section 12.3). A program that receives snapshots needs them typed, which soundness by typing guarantees step by step (Section 12.4).

12.1 Patching expressions

An expression is a tree, so an update can patch a computation before it runs, as it patches a program (Section 9.3). For example, the expression q @ d is the tree Fk (Sm q) (Sm d) and an update receives it without evaluating it. Call such an expression, an application not yet evaluated, *pending*.

Theorem 50 (update_preserves_expression_typing). If ontology is sound by values and u : Update and e :: T and u @ e $\Longrightarrow$ e' then e' :: T.

The value type of an expression is built by the concrete clauses from the types of its parts. It determines the expression type (expression_type_factors). So an update, which preserves the value type, preserves the expression type too. Taken as a value, the tree q @ d has type Q @ D when q : Q and d : D. An example is Query[List[Emp]] @ Dept for the pending query below.

For example, the salary update patches both the query and the data of the pending application paid_over[57] @ department_2025 to give paid_over[58]@department_2026 and preserves its expression type List[Emp] (patched_query_typed, salary_update_pending_query_typed).

For this query and this update, patching and evaluation commute: patching the result of the expression gives the same as evaluating the patched expression. Write L_{57} = [emp[1, 58], emp[3, 57]; emp[0, 0]] and L_{58} = [emp[1, 59], emp[3, 58]; emp[0, 0]].

Theorem 51 (patch_commutes_with_evaluation). Patching then evaluating gives L_{58}:

salary_update @ (paid_over[57] @ department_2025)
  $\Longrightarrow$ paid_over[58] @ department_2026
paid_over[58] @ department_2026 $\Longrightarrow$ L_{58}.

Evaluating then patching gives L_{58} too:

paid_over[57] @ department_2025 $\Longrightarrow$ L_{57}
salary_update @ L_{57} $\Longrightarrow$ L_{58}.

12.2 Typed optimisation

Updates preserve the value type of their argument, whereas an optimisation need preserve only its expression type. Typing leaves its mark on programs: every value of an abstract type is tagged at its root (infer_tagged_result_inv), and lambda programs and some data values carry ghosts. Once an expression has been typed, these have done their work, but they still cost steps. A typed optimiser erases them before evaluation.

An abstract program that is applied behaves as its carrier: since the label is an identity, tagged[t, n] @ v and t @ v evaluate alike (tagged_app_eval_iff). The program optimise replaces the operator of each application in an expression by its carrier, repeatedly if the carrier is itself tagged, and then replaces a decoration decorate[f, w] by f because decorate[f, w] @x evaluates f @x (decorate_app_eval_iff). Only the operators of applications change. It recognises an abstract program by the shape test get_tagged of Section 2.5, whatever its nametag, so one program serves every abstract type, function types included. It contains no labels, as infer does not, so it is concrete and hence self-typed (optimise_eval, optimise_concrete).

Theorem 52 (optimise_preserves_evaluation). If optimise @ e $\Longrightarrow$ e' then e $\Longrightarrow$ v if and only if e' $\Longrightarrow$ v.

Rocq states this for the function that optimise computes. The two statements agree, since optimise @ e evaluates to the result of that function (optimise_eval) and evaluation is deterministic.

Preservation of expression types needs soundness by typing. Suppose that an abstract program of type tagged[E, N] is applied to an argument of type U with result type V. If the program's carrier has type C then soundness by typing (Section 3.3) gives C # U $\Longrightarrow$ Sm V. So the optimised application has the same type. Call a value in the operator position of an application in an expression, at any depth, a *head* of the expression.

A head is *unsafe* if, after the removal of any number of its tags, it is tagged with query_nm, update_nm or is_named_nm and its carrier is (a) a name test, of the form x ∘ get_tagname; (b) of the form of the conditional if_named; or (c) a fork whose left branch is a fork, such as a triage. Every other head is *safe*. The program heads_avoid checks that every head of an expression is safe. The unsafe shapes cover the carriers of the triage query, the children update, the specialised queries and updates, and IsNamed. At their types the abstractors of paper_ontology are sound by values only (Sections 6, 8 and 9). Erasing the tag of an unsafe head can leave an untypable application. The triage

Program	Before	After
infer with paper_ontology	70069	97721
paper_ontology	66858	94510
get_IfNamed	3424	17250
query_abstractor[Lf]	15963	29789
update_abstractor[Lf]	9872	23698
specialise_equal	1224	1224

Steps to a value of	Before	After
infer @ 1	132996	132996
infer @ emp[1, 2]	755366	755366
infer @ salary_bill	23680101	23620501
infer @ salary_update	15297806	15238206

Figure 22. Sizes before and after specialise_equal and exact step counts of inference

query triageq[tt, Kq[tt], Kq[Kq[tt]]] applied to tt shows this: for paper_ontology it has an expression type and an unsafe head, and its optimised form has no type (optimise_unsafe_head_untyped). So the theorem asks for safe heads.

For example, optimise turns Kq[0] @ tt into $K_1 0$ @ tt by erasing the query tag. On a lambda program it erases the function tag, and with it the ghost (Section 10).

Theorem 53 (optimise_typed).
For paper_ontology: (1) optimise : optimise; (2) optimise @ (Kq[0] @ tt) $\Longrightarrow$ $K_1 0$ @ tt; and (3) if $e :: T$ and every head of e is safe and optimise @ $e \Longrightarrow e'$ then $e' :: T$.

Since tags are the handles by which queries and updates find their targets, erasing them should come last, after any update.

A fixfunction is a value, so optimise does not reach inside it. A further optimisation erases the tags and ghosts of the functions composed inside the body b of a recursive function fix_function[lam[b, w]] of type $U \to V$. The erased function keeps the type $U \to V$ exactly when the type of the erased b maps $U \to V$ to $U \to V$ (fix_erase_typing_iff).

12.3 Sizes and the trade between space and time

Because size is a program, measurement happens inside the calculus: size @ p evaluates to the numeral of the number of nodes of p (size_eval). So a choice between space and time can be measured, and made, by a typed program. The example below, *specialising equality*, walks the ontology. The ontology after the walk is self-typed, and its soundness is vouched for, not proved.

Theorem 54 (size_table).
For each row of the first table of Figure 22, size gives the first number on the program p of that row; specialise_equal @ p evaluates to some p'; and size gives the second number on p'.

By the first two rows of Figure 22, all but 3211 of the nodes of infer belong to the ontology. Comparisons with closed values are one place to trade space for time. The test equal@c evaluates to a value equal1[c] that compares its argument with c and is called the *tester* of c (equal_eval). The size of the tester grows with c. The value wait[equal, c] holds equal and c apart, and builds the tester only when it is applied.

Theorem 55 (tester_agrees).
wait[equal, c] @ $x \Longrightarrow v$ if and only if equal1[c] @ $x \Longrightarrow v$.

Pattern matching compiles a comparison with a closed value c of size at most 16 to equal1[c] and one with a larger value to wait[equal, c] (closed_test). In specialising equality, the program specialise_equal walks a tree and replaces each wait[equal, c] by the value of equal@c and so evaluates each such test once, in advance. It leaves wait[equal, c] alone when c is a stem, and it does not descend into c (specialise_equal_eval).

The walk finds four sites in all, two in each of the query and update abstractors of infer. They lie in the abstractors' copies of the program get_IfNamed that recognises the carrier type IfNamed[n, F, D]. The walk of get_IfNamed itself finds the same two (specialise_sites_get_IfNamed). In each abstractor the two values c have 324 and 190 nodes, and their testers have 9272 and 5744 (specialise_sites). The program get_IfNamed and each abstractor grow by the same 13826 nodes. In Figure 22 the abstractors are measured with the leaf as their ontology. By the first row, infer grows from 70069 to 97721 nodes, by twice 13826, or 39%. In the examples of the figure, typing saves 59600 steps when it reaches a specialised query or update, and none otherwise. Here the trade is poor: 39% more space saves under half of one percent of the steps. Yet it is measured, and made, by typed programs.

The walk preserves concreteness, since equal1[c] holds no label for any c (specialise_concrete). So a concrete program stays self-typed (specialise_self_typed), including infer built using the ontology after the walk on paper_ontology (specialised_infer_self_typed).

Types of expressions need a condition. Write heads_only e when every wait[equal, c] that the walk replaces in e is the operator of an application with c tag-free, and when the walk leaves every other value of e unchanged. At the type level, equal # $c \Longrightarrow$ Sm equal1[c] for tag-free c (app_equal), so the two forms apply alike at every argument type.

Theorem 56 (specialise_typed).
For every ontology: (1) specialise_equal : specialise_equal; (2) specialise_equal @ wait[equal, $K_1 c$] $\Longrightarrow$ equal1[$K_1 c$]; and (3) if heads_only e and $e :: T$ and specialise_equal @ $e \Longrightarrow e'$ then $e' :: T$.

The condition is needed. The value wait[equal, K_1Lf] is self-typed, and the walk changes its type (specialise_changes_type). And if the walk replaced the comparison inside the name test of is_named[n] then the result would have no type at all (specialised_name_test_untyped), because is_named_abstractor reads name_test[N] from the carrier

type. The name test name_test[n] holds wait[equal, Sm n] whose value c is a stem, so the walk leaves stems alone.

An optimisation must preserve typing, and may change results. An intensional program such as size will and must see a difference, as Figure 22 shows. It is on the programmer to vouch for the optimisation, or to prove that the result is unchanged because the program is extensional where it matters. For the tester, that proof is Theorem 55. Both paper_ontology and the ontology after the walk are shown to support function types, booleans, numbers, options, sums, lists, arrays, salaries, employees, departments, seeks and IsNamed. So anything proved from that support holds of both (`unchanged_from_support`). Elsewhere the result is vouched for.

12.4 Snapshots of evaluation

A computation in progress is just an expression, so programs can receive the snapshots of evaluation as arguments, to measure a computation or to repair it without starting again.

From Figure 3 comes a program that performs one step at a time, where a step applies one rule of the figure or evaluates a name test in full. A name test is evaluated in full because its intermediate states need not be typed. If a step applied only one rule inside a name test then the first snapshot of the typed expression is_named[bool_nm]@tt would be untyped under an ontology of booleans and IsNamed (`name_test_snapshot_untyped`). Some number of steps takes e to Sm v exactly when $e \Longrightarrow v$ (`steps_agree`). Write snapshot[j] e for the expression that e reaches after j steps, its *snapshot*.

If ontology is sound by typing then each step preserves expression types (`step_preserves_expression_typing`). The proof uses soundness by typing, since a step can expose the carrier of a query or update that takes its argument apart. A snapshot can be untyped for paper_ontology since it is sound by values only. The expression size @ tt has an expression type, but its snapshot after 165 steps has none, since it exposes the triage carrier of size at the abstract type Bool (`size_snapshot_untyped`).

The next results instead assume that a snapshot is typed, so they do not need soundness by typing. If ontology is sound by values then an update can patch a typed snapshot, keeping its type (`patch_during_evaluation`). If it supports queries and numbers then size applies to a typed snapshot (`size_on_snapshots`).

Counting steps measures time, and the size of a snapshot measures space. The program loop passes the negation of its argument to its recursive call. Its snapshots cycle, so their sizes are bounded (`loop_cycle` and `loop_size_bounded`). The snapshots of dither @ tt instead gain 86 nodes every 92 steps, each round holding one more pending negation. If ontology supports queries and booleans then the snapshot at the end of each round has type Bool (`dither_pending_typed`). The difference lies in where the negation sits: the loop negates its argument before its recursive call, whereas dither negates the result of its recursive call, which must wait.

Theorem 57 (`size_on_expressions`).
(1) size@snapshot[$92k$] (dither@tt) $\Longrightarrow 339+86k$ for every k; and (2) for paper_ontology if $e :: T$ then size @ e :: Nat.

Minimisation with function types (Section 10.2) appears here too. Consider the search by minimisation_fn from 0 for a number equal to its successor. Its snapshot at the call of the predicate at 0 has 4426 nodes. Its snapshot at the call at 3 has $4426 + 150 \cdot 3$ nodes (`search_call_0`, `search_call_3`). These are profiles of particular examples, but they hint at a complexity theory within the calculus, with step counting as the clock and size as the measure.

13 Related Work

Typing combinators. Traditional accounts type combinatory logic through the lambda calculus. Hindley and Seldin assign K the family of types $U \to V \to U$ with substitution supplying its instances [18], while System F gives it the quantified type $\forall X.\forall Y.\, X \to Y \to X$ [15]. In both, the polymorphism comes from type variables and their binding. Here $K : K$ and its polymorphism is revealed by how its type applies (Section 4). The closest precursor assigns simple types to polymorphic tree-calculus functions [24], and an earlier typing of program analysis in tree calculus uses quantified types [22]. The present paper makes type application and inference programs, with an extensible ontology of abstract types.

Church and Curry. Church and Curry differ over whether there can be bad programs [8, 12]. For Church, a term is typed by construction; for Curry, typing is a property that some terms lack. Tree calculus takes Curry's view of programs: every tree is a program, and inference may fail on it. The same question arises for types. Traditional systems have formation rules, so that only well-formed types exist. Here types follow Curry too: any value can serve as a type, and type application at it may simply fail. Requiring each abstractor to be sound, for every value of every argument type, makes this safe. For typing à la Church and à la Curry, see [5].

Analysing programs. In the lambda-calculus tradition, analysing an arbitrary program, as by computing its size, requires a representation of the program as data: quoted syntax, a typed self-representation [6, 28], or the staged code of multi-stage programming [35]. Tree calculus analyses programs as they stand [21], and here the analyses are typed. Generic programming over a universe of datatype descriptions [19] treats the datatypes of that universe uniformly, and a parametric type such as $\forall A.\, A \to \text{Nat}$ cannot inspect its argument at all [31, 39]. Here size has the query type Query[Nat] of Section 8 and inspects every value, including functions and the inference program.

Intensional type analysis. Typecase and intensional type analysis let a program branch on a type passed at run time [16]. The type-directed conditionals of Section 6 branch instead on the argument's own nametag. Their typing uses the family IsNamed[n] of the ontology. Its type application computes the answer of the test from the argument type. Occurrence typing [37, 38] instead refines the type of a tested variable within each branch. Here no variable is refined, and the type of the conditional is that of the arm the test selects. The conditionals that specialise queries and updates act at wrapped types, which have a name of their own, as with a newtype.

Generic queries and updates. "Scrap your boilerplate" [26] builds generic queries and updates in Haskell from a type-safe cast and one-layer traversals, supported by derived type-class instances for each datatype. The pattern calculus [20] supports generic queries and updates by path polymorphism, with types that quantify over the structures traversed. Here queries and updates are abstract types of their own, whose constructors are proved to work on abstract values that type application refuses to inspect (Sections 8 and 9), and the same updates act on data, programs and pending expressions.

Meta-theory. The meta-theory of tree calculus is small. Its big-step semantics has ten rules with no side conditions, and evaluation is deterministic (Theorem 1). Typing is then programming within the calculus. Determinism does not fix a strategy. The evaluation here is branch-first (eager). Branch-first, root, root-and-branch and root-first evaluation are all programs in the calculus [21, Section 6.2], as typing is. Type comparison is the program equal where pure type systems [4] use a meta-level equivalence.

Computation in types. Languages with dependent types, such as Cayenne [3], compute types by general programs, so type checking need not terminate. Inference here is likewise a program that may diverge (Section 3.2). Length-indexed arrays compute their lengths by type application on concrete indices, so type agreement is equality of trees (Section 7). In a pure type system [4] that agreement is conversion, a relation of the meta-theory. Programming conversion for dependent types is future work (Appendix A). Concrete types resemble singleton types [2], in that each has a single value, but they arise from the concrete clauses rather than from a type former.

Types as abstract interpretation. Cousot presents type systems as abstract interpretations of the semantics [10, 11]. Here the concrete clauses of type application mirror evaluation, and the abstract clauses forget how values of abstract types are built, so the two kinds of type are the two extremes of forgetting within one program.

Self-typing. The self-typing of infer (Figure 2) is an instance of a theorem about concrete programs (Section 4): each concrete program is its own type, with itself as its only value. It is related to Type : Type but is not an instance of it. A type of all types is inconsistent once types can be quantified over [14], although Type : Type remains useful in programming languages [7]. Types here are not quantified over, so a type of all types remains a possible extension. Self types [13, 34] are something else. They are self-references, as is the type of self in object calculi [1]: a type refers to the term or object it describes. In Cedille, for example, the type of numbers states induction for the number at hand. Self types resemble the type Nat here, whose type application describes how a number is folded, more than the self-typing of infer.

14 Conclusions

This paper demonstrates that typing rules can become programs in tree calculus. Types and terms are computed in the same calculus. Each typing rule becomes a clause of the program for type application or type inference. Whether a program is run or read depends on its role in an application. Function types become an afterthought, as type application does the work of quantifiers and type variables.

Inference takes as a parameter the ontology of abstract types, and evaluation preserves typing whenever the ontology is sound. One sound ontology paper_ontology supports general recursion, polymorphic lists, type-directed conditionals, length-indexed arrays, and generic and specialised queries and updates that act on data, programs and expressions. General recursion is typed in two ways. Dedicated invariants, the types of numbers, queries and updates, suffice for the main examples, including typed partial recursion on numbers, and generic queries and updates. Uniform support is provided by function types. Since abstract types do not usually have types, they cannot be passed as parameters. Instead, values and nametags act as proxies for types when instantiating polymorphism or introducing type-directed conditionals. This is fragile but might be improved by introducing a type of all types Type.

The concrete programs are self-typed, and each concrete type has only one value, namely itself. They include paper_ontology and the program infer built on it, which is then self-typed. That said, applications of this infer need not be typed. Perhaps a Type-valued query would be more useful.

More generally, since programs, the expressions that run them, and the snapshots of evaluation are all trees, tools can be typed programs that act on expressions. Future work will consider other type systems, such as dependent types and pure type systems. In turn, this fits within the larger framework of Project Leibniz (Appendix A). Even though

there is more programming to be done, it is clear that typing rules have become programs in tree calculus.

References

[1] Martín Abadi and Luca Cardelli. 1996. *A Theory of Objects*. Springer. https://doi.org/10.1007/978-1-4419-8598-9

[2] David Aspinall. 1994. Subtyping with Singleton Types. In *Computer Science Logic (CSL '94) (Lecture Notes in Computer Science, Vol. 933)*. Springer, 1–15.

[3] Lennart Augustsson. 1998. Cayenne – a Language with Dependent Types. In *Proceedings of the Third ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 239–250.

[4] Henk P. Barendregt. 1991. Introduction to Generalized Type Systems. *Journal of Functional Programming* 1, 2 (1991), 125–154. https://doi.org/10.1017/S0956796800020025

[5] Henk P. Barendregt. 1992. Lambda Calculi with Types. In *Handbook of Logic in Computer Science*, Samson Abramsky, Dov M. Gabbay, and Thomas S. E. Maibaum (Eds.). Vol. 2. Oxford University Press, 117–309. https://doi.org/10.1093/oso/9780198537618.003.0002

[6] Matt Brown and Jens Palsberg. 2016. Breaking through the Normalization Barrier: A Self-Interpreter for F-omega. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 5–17.

[7] Luca Cardelli. 1986. *A Polymorphic λ-calculus with Type:Type*. SRC Research Report 10. DEC Systems Research Center.

[8] Alonzo Church. 1940. A Formulation of the Simple Theory of Types. *Journal of Symbolic Logic* 5, 2 (1940), 56–68.

[9] Thierry Coquand and Gérard Huet. 1988. The Calculus of Constructions. *Information and Computation* 76, 2–3 (1988), 95–120. https://doi.org/10.1016/0890-5401(88)90005-3

[10] Patrick Cousot. 1997. Types as Abstract Interpretations. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 316–331.

[11] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. 238–252.

[12] Haskell B. Curry and Robert Feys. 1958. *Combinatory Logic, Volume I*. North-Holland.

[13] Peng Fu and Aaron Stump. 2014. Self Types for Dependently Typed Lambda Encodings. In *Rewriting and Typed Lambda Calculi, Joint International Conference (RTA-TLCA 2014) (Lecture Notes in Computer Science, Vol. 8560)*. Springer, 224–239. https://doi.org/10.1007/978-3-319-08918-8_16

[14] Jean-Yves Girard. 1972. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Ph. D. Dissertation. Université Paris VII.

[15] Jean-Yves Girard, Yves Lafont, and Paul Taylor. 1989. *Proofs and Types*. Cambridge Tracts in Theoretical Computer Science, Vol. 7. Cambridge University Press.

[16] Robert Harper and Greg Morrisett. 1995. Compiling Polymorphism Using Intensional Type Analysis. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 130–141.

[17] Fritz Henglein. 1993. Type Inference with Polymorphic Recursion. *ACM Transactions on Programming Languages and Systems* 15, 2 (1993), 253–289.

[18] J. Roger Hindley and Jonathan P. Seldin. 2008. *Lambda-Calculus and Combinators: An Introduction*. Cambridge University Press. https://doi.org/10.1017/CBO9780511809835

[19] Ralf Hinze. 2000. A New Approach to Generic Functional Programming. In *Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 119–132. https://doi.org/10.1145/325694.325709

[20] Barry Jay. 2009. *Pattern Calculus: Computing with Functions and Structures*. Springer.

[21] Barry Jay. 2021. Reflective Programs in Tree Calculus. https://github.com/barry-jay-personal/tree-calculus.

[22] Barry Jay. 2025. Typed Program Analysis without Encodings. In *Proceedings of the 2025 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation*. https://doi.org/10.1145/3704253.3706138

[23] Barry Jay. 2026. Typing Rules Become Programs in Tree Calculus: Rocq Development. https://github.com/barry-jay-personal/typing-rules-as-programs.

[24] Barry Jay and Johannes Bader. 2026. Simple Types for Polymorphic Functions. *arXiv preprint arXiv:2604.12194* (2026).

[25] Saul Kripke. 1975. Outline of a Theory of Truth. *Journal of Philosophy* 72, 19 (1975), 690–716.

[26] Ralf Lämmel and Simon Peyton Jones. 2003. Scrap Your Boilerplate: A Practical Design Pattern for Generic Programming. In *Proceedings of the ACM SIGPLAN Workshop on Types in Language Design and Implementation (TLDI)*. 26–37.

[27] Robin Milner. 1978. A Theory of Type Polymorphism in Programming. *J. Comput. System Sci.* 17, 3 (1978), 348–375. https://doi.org/10.1016/0022-0000(78)90014-4

[28] Torben Æ. Mogensen. 1992. Efficient Self-Interpretation in Lambda Calculus. *Journal of Functional Programming* 2, 3 (1992), 345–364.

[29] G.D. Plotkin. 2004. A Structural Approach to Operational Semantics. (2004). http://homepages.inf.ed.ac.uk/gdp/publications/sos_jlap.pdf.

[30] Willard Van Orman Quine. 1940. *Mathematical Logic*. Harvard University Press, Cambridge, MA. Section 4, "Use versus mention".

[31] John C. Reynolds. 1983. Types, Abstraction and Parametric Polymorphism. In *Information Processing 83*. North-Holland, 513–523.

[32] Bertrand Russell. 1908. Mathematical Logic as Based on the Theory of Types. *American Journal of Mathematics* 30, 3 (1908), 222–262.

[33] Moses Schönfinkel. 1924. Über die Bausteine der mathematischen Logik. *Math. Ann.* 92 (1924), 305–316.

[34] Aaron Stump. 2017. The Calculus of Dependent Lambda Eliminations. *Journal of Functional Programming* 27 (2017), e14. https://doi.org/10.1017/S0956796817000053

[35] Walid Taha and Tim Sheard. 2000. MetaML and Multi-Stage Programming with Explicit Annotations. *Theoretical Computer Science* 248, 1–2 (2000), 211–242. https://doi.org/10.1016/S0304-3975(00)00053-0

[36] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, NJ. https://homotopytypetheory.org/book/

[37] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 395–406. https://doi.org/10.1145/1328438.1328486

[38] Sam Tobin-Hochstadt and Matthias Felleisen. 2010. Logical Types for Untyped Languages. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 117–128. https://doi.org/10.1145/1863543.1863561

[39] Philip Wadler. 1989. Theorems for Free!. In *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. ACM, 347–359. https://doi.org/10.1145/99370.99404

[40] Hongwei Xi and Frank Pfenning. 1999. Dependent Types in Practical Programming. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 214–227. https://doi.org/10.1145/292540.292560

A The Leibniz Project

Leibniz envisioned a single calculus for reasoning. Conventional attempts are fragmented by type hierarchies or encodings. Tree calculus keeps programs, types and their analyses on one level, as trees analysed where they stand. This paper is one stage of a longer project to pursue that vision. Its milestones are self-applications, beginning with the evaluator eval of tree calculus [21]:

eval @ eval
infer @ infer
compile @ compile
cost @ cost.

The first two have been reached, and the other two are the next milestones. Figure 23 shows how the stages depend on one another.

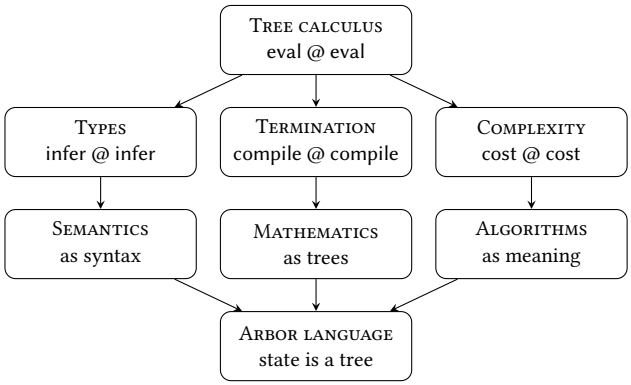


Figure 23. The Leibniz Project

A.1 What has been done

In earlier work, tree calculus has been developed and verified in Rocq [21]. Its evaluator eval acts on programs as they stand, so that eval @ eval is a self-evaluation. Tree calculus can also be typed in the traditional way [22]. A precursor of the present approach treats combinatory and abstract types in a rewriting system [24]. This paper represents types and type inference as programs, with the contributions listed in Section 1.

A.2 What remains

Types. Dependent types, and pure type systems in general, remain to be recast as programming, beyond the fragment shown by array append and perhaps the type-directed conditionals of Section 6, whose types choose by the argument's type. So do proofs as programs, and the proof of EPT within the calculus itself, rather than in Rocq. For proofs, a judgement $w : p$ with p : Prop would make a proposition both a value and the type of its proofs; families of type $A \to$ Prop would require a combinatory account of dependent function types; and logical consistency would be a further obligation, which EPT alone does not establish. A related challenge is to type the applications of the inference program. Even though infer : infer holds, typing the application infer @ u faces two obstacles. First, type application of the type of infer refuses abstract argument types (Section 11). Second, its result is a type, and in general a type has no type, as Bool has none (`Bool_untypable`). So a tool that consumes the results of inference, such as a compiler or a cost analysis, needs a typed counterpart of inference. The challenge is to give one without introducing a type of all types. One candidate is a query type: since infer reads any argument, as a query does, it could also be given a type Query$[B]$ for a suitable type B of its results. That would meet the first obstacle, and leave the choice of B as the second. The aim is semantics as syntax, in which substantial parts of the meta-theory of computing and of logic become programs. Each of these is a research problem in its own right.

Termination. The plan is an analytic subsystem in which all computations terminate, yet which supports primitive recursion and the generic queries and updates needed to analyse and transform arbitrary trees. It should culminate in a compiler, expressed as a program of the calculus, whose self-application compile @ compile tests whether the subsystem is closed under its own compilation. The typed optimiser of Section 12.2 is a first step. The subsystem must be weak enough to guarantee termination, yet strong enough to contain its own compiler. Whether both can be had at once is open. Typing raises the same question. Expression typing need not terminate, as for ω @ ω. So a first task is to find conditions that guarantee termination of infer.

Complexity. Since size is a generic query, it can measure the inputs of higher-order programs directly, and the snapshots of their evaluation (Section 12.4). Each step preserves the typing of snapshots when the ontology is sound by typing. The ontology of this paper is sound by values only, and some of its snapshots are untyped (Section 12.4). It is open which snapshots stay typed, and whether a coarser step keeps them all typed. Measuring snapshots is a first step towards a native complexity theory that infers costs as type inference infers types, and that relates the size of input trees to reduction time, space and the growth of intermediate trees. The cost of a higher-order program depends on the costs of its arguments, and cost @ cost is the milestone that would capture this.

Further. If these stages succeed, three larger ones follow. Mathematics could be rebuilt constructively, in the style of Bishop, using trees rather than sets and logic. The reason to use trees is that Russell's hierarchy of types blocks vicious circularity but excludes virtuous self-reference too, whereas tree calculus allows self-application without hierarchy. Algorithms could be recast over trees, so that process, cost and intermediate structure become part of their meaning. Finally,

these theories could converge on Arbor, a programming language whose entire state, including code, stack and heap, is a tree. Each of these is a larger undertaking than everything described above.